HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



MÔN HỌC: IOT VÀ ỨNG DỤNG

BÁO CÁO THỰC HÀNH 1

Giảng viên hướng dẫn	: TS. Nguyễn Quốc Uy
Nhóm lớp học phần	: 10
Họ và tên	: Nguyễn Quang Huy
Mã sinh viên	: B23DCCN398

Hà Nội, 2026

MỤC LỤC

MỤC LỤC 1

I. GIỚI THIỆU 4

1.1. Mục đích 4

1.2. Phạm vi hệ thống 4

II. THIẾT KẾ TỔNG QUAN HỆ THỐNG 5

2.1. Thành phần chính của hệ thống 5

2.1.1. Thành phần Frontend 5

2.1.2. Thành phần Backend 6

2.1.3. Database 6

2.1.4. MQTT Broker 7

2.1.5. Hardware 7

2.2 Thiết kế cơ sở dữ liệu 7

2.2.1 Sơ đồ ERD 7

2.2.2. Bảng users 8

2.2.3. Bảng sensors 9

2.2.4. Bảng devices 9

2.2.5. Bảng sensor_data 9

2.2.6. Bảng action_history 10

III. THIẾT KẾ CHI TIẾT 11

3.1. Tổng quan use case 11

3.1.1. Actor 11

3.1.2. Danh sách Use Case 11

3.2 Đặc tả chi tiết use case 11

3.2.1. UC01 – Login 11

a. Đặc tả use case 11

b. Sequence diagram 12

c. API Docs 13

d. Mapping dữ liệu đăng nhập 13
e. Hình ảnh giao diện 14
3.2.2. UC02 – View Realtime Sensor Data 14
a. Đặc tả use case 14
b. Luồng khi User vừa mở trang Dashboard 14
c. Luồng cảm biến đo lại giá trị sau mỗi khoảng thời gian 16
d. Hình ảnh giao diện 17
3.2.3. UC03 – View Sensor Chart 17
a. Đặc tả use case 17
b. Luồng khi User vừa mở trang Dashboard 17
c. Luồng khi cảm biến đo được dữ liệu mới 19
d. Hình ảnh giao diện 21
3.2.4. UC04 – Control Device 21
a. Đặc tả use case 21
b. Sequence diagram 23
c. API docs 23
d. Mapping dữ liệu 24
e. Hình ảnh giao diện 25
3.2.5. UC05 – View/Search Sensor History 25
a. Đặc tả use case 25
b. Sequence diagram 26
c. API docs 26
d. Mapping dữ liệu 27
e. Hình ảnh giao diện 28
3.2.6. UC06 – View/Search Device Action History 28
a. Đặc tả use case 28
b. Sequence diagram 29
c. API docs 29

d. Mapping dữ liệu 30
e. Hình ảnh giao diện 31
IV. KẾT QUẢ THỰC HIỆN 32
4.1. Kết quả triển khai hệ thống 32
a. Triển khai phần cứng và chương trình trên ESP8266 32
b. Triển khai MQTT Broker 33
c. Khởi tạo và tổ chức cơ sở dữ liệu MySQL 33
d. Triển khai Backend bằng Java Spring Boot 34
e. Triển khai giao diện Web bằng HTML, CSS và JavaScript 34
4.2. Kết quả giao diện và chức năng 34
a. Màn hình đăng nhập 35
b. Dashboard - dữ liệu thời gian thực, biểu đồ và điều khiển đèn 35
c. Màn hình Sensor Data 36
d. Màn hình Device Action History 37
e. Màn hình Profile 38
4.3. Kết quả kiểm thử 38
a. Kiểm thử API đăng nhập - POST /api/auth/login 38
b. Kiểm thử API lấy dữ liệu cảm biến mới nhất - GET /api/sensor-data/latest 39
c. Kiểm thử API dữ liệu biểu đồ - GET /api/sensor-data/chart 40
d. Kiểm thử API lấy trạng thái thiết bị - GET /api/devices/status 41
e. Kiểm thử API điều khiển đèn - POST /api/devices/{deviceId}/control 42
f. Kiểm thử API lịch sử cảm biến - GET /api/sensor-data/history 43
g. Kiểm thử API lịch sử điều khiển - GET /api/device-action/history 44
V. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 47
5.1. Kết quả đạt được 47
5.2. Hạn chế của hệ thống 47
5.3. Hướng phát triển 47
5.4. Kết luận 48

I. GIỚI THIỆU

1.1. Mục đích

Hệ thống cho phép người dùng theo dõi dữ liệu được gửi từ các cảm biến trong môi trường theo thời gian thực, xem biểu đồ dữ liệu, tra cứu lịch sử cảm biến, điều khiển và kiểm tra lịch sử bật/tắt các thiết bị đèn.

1.2. Phạm vi hệ thống

Hệ thống được xây dựng gồm các thành phần chính:

- Giao diện Web dành cho người dùng.
- Backend cung cấp REST API và xử lý nghiệp vụ.
- Cơ sở dữ liệu dùng để lưu thông tin người dùng, cảm biến, thiết bị, dữ liệu cảm biến và lịch sử điều khiển.
- MQTT Broker làm trung gian trao đổi dữ liệu giữa Backend và phần cứng IoT.
- Phần cứng IoT có nhiệm vụ thu thập dữ liệu cảm biến và thực hiện lệnh điều khiển thiết bị.

Các chức năng chính của hệ thống gồm:

1. Đăng nhập vào hệ thống.
2. Xem dữ liệu cảm biến theo thời gian thực.
3. Xem biểu đồ dữ liệu cảm biến.
4. Bật/tắt thiết bị.
5. Xem và tìm kiếm lịch sử dữ liệu cảm biến.
6. Xem và tìm kiếm lịch sử bật/tắt thiết bị.
7. Xem thông tin cá nhân và các liên kết liên quan đến dự án.

Trong phạm vi bài tập, hệ thống sử dụng ba loại cảm biến:

- Cảm biến nhiệt độ.
- Cảm biến độ ẩm.
- Cảm biến ánh sáng.

Hệ thống điều khiển hai thiết bị đèn:

- Đèn 1.
- Đèn 2.

II. THIẾT KẾ TỔNG QUAN HỆ THỐNG

2.1. Thành phần chính của hệ thống

Các thành phần chính của hệ thống:

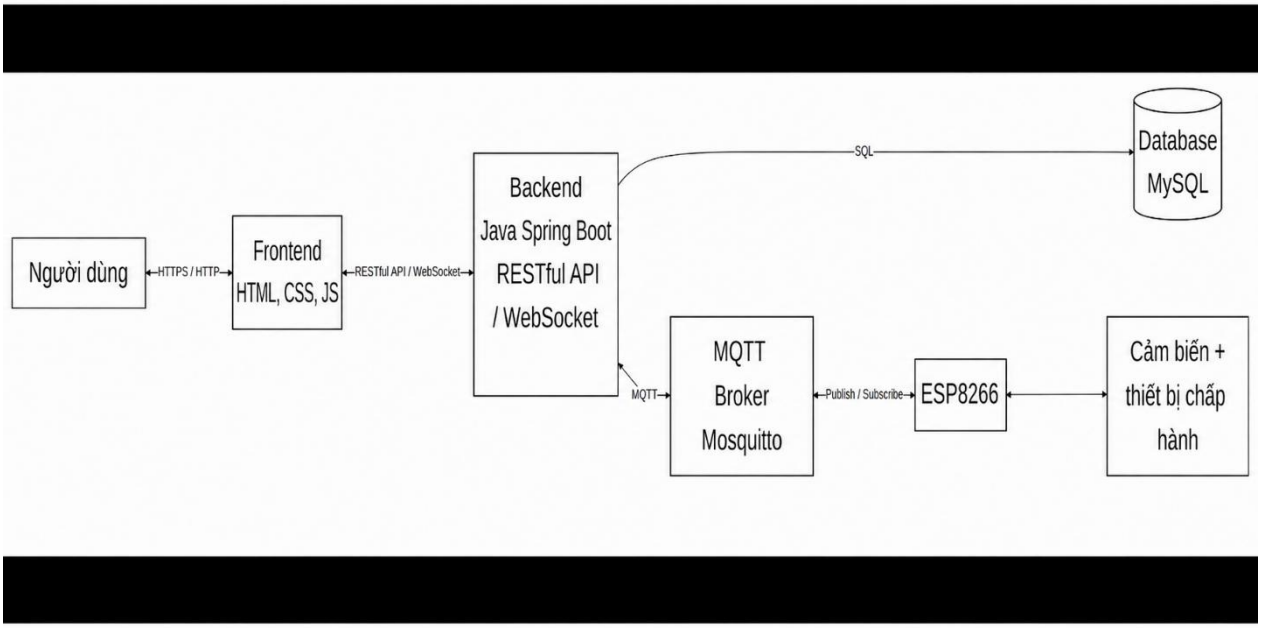
- User.
- Frontend Web.
- Backend.
- Database.
- MQTT.
- Hardware.

Luồng tổng quát:

User → Frontend → Backend → Database/MQTT → Hardware

Phần cứng gửi dữ liệu cảm biến theo chiều ngược lại:

Hardware(Sensor) → MQTT → Backend → Database → Frontend → User



Hình 2.1: Tổng quan hệ thống

2.1.1. Thành phần Frontend

Frontend là giao diện Web mà người dùng trực tiếp sử dụng.

Frontend thực hiện các chức năng:

- Hiển thị Dashboard.
- Hiển thị dữ liệu cảm biến.
- Hiển thị biểu đồ.
- Gửi yêu cầu bật/tắt thiết bị.
- Gửi yêu cầu tìm kiếm lịch sử.
- Hiển thị trạng thái phản hồi của thiết bị.
- Hiển thị Profile.

Công nghệ sử dụng: HTML, CSS, JS

2.1.2. Thành phần Backend

Backend chịu trách nhiệm:

- Xác thực người dùng.
- Cung cấp REST API.
- Nhận yêu cầu từ Frontend.
- Đọc và ghi dữ liệu trong Database.
- Publish message đến MQTT Broker.
- Subscribe các MQTT topic.
- Nhận dữ liệu cảm biến.
- Nhận phản hồi trạng thái từ thiết bị.
- Trả dữ liệu cho Frontend.

Công nghệ sử dụng: Java Spring Boot

2.1.3. Database

Database lưu trữ các dữ liệu có tính lâu dài của hệ thống, bao gồm:

- Người dùng.
- Danh sách cảm biến.
- Danh sách thiết bị.
- Dữ liệu cảm biến.
- Lịch sử điều khiển thiết bị.

Hệ quản trị cơ sở dữ liệu: MySQL

2.1.4. MQTT Broker

MQTT Broker là thành phần trung gian giữa Backend và phần cứng IoT.

MQTT được sử dụng để:

- Backend gửi lệnh bật/tắt thiết bị.
- Hardware nhận lệnh điều khiển.
- Hardware gửi phản hồi trạng thái.
- Hardware gửi dữ liệu cảm biến.
- Backend nhận dữ liệu và phản hồi từ phần cứng.

MQTT giúp hệ thống thực hiện giao tiếp hai chiều giữa phần mềm và thiết bị IoT.

2.1.5. Hardware

Phần cứng có nhiệm vụ:

- Đọc dữ liệu nhiệt độ.
- Đọc dữ liệu độ ẩm.
- Đọc dữ liệu ánh sáng.
- Gửi dữ liệu lên MQTT Broker.
- Nhận lệnh điều khiển đèn.
- Bật/tắt thiết bị.
- Gửi trạng thái thực tế của thiết bị trở lại hệ thống.

Phần cứng sử dụng: ESP8266

2.2 Thiết kế cơ sở dữ liệu

2.2.1 Sơ đồ ERD

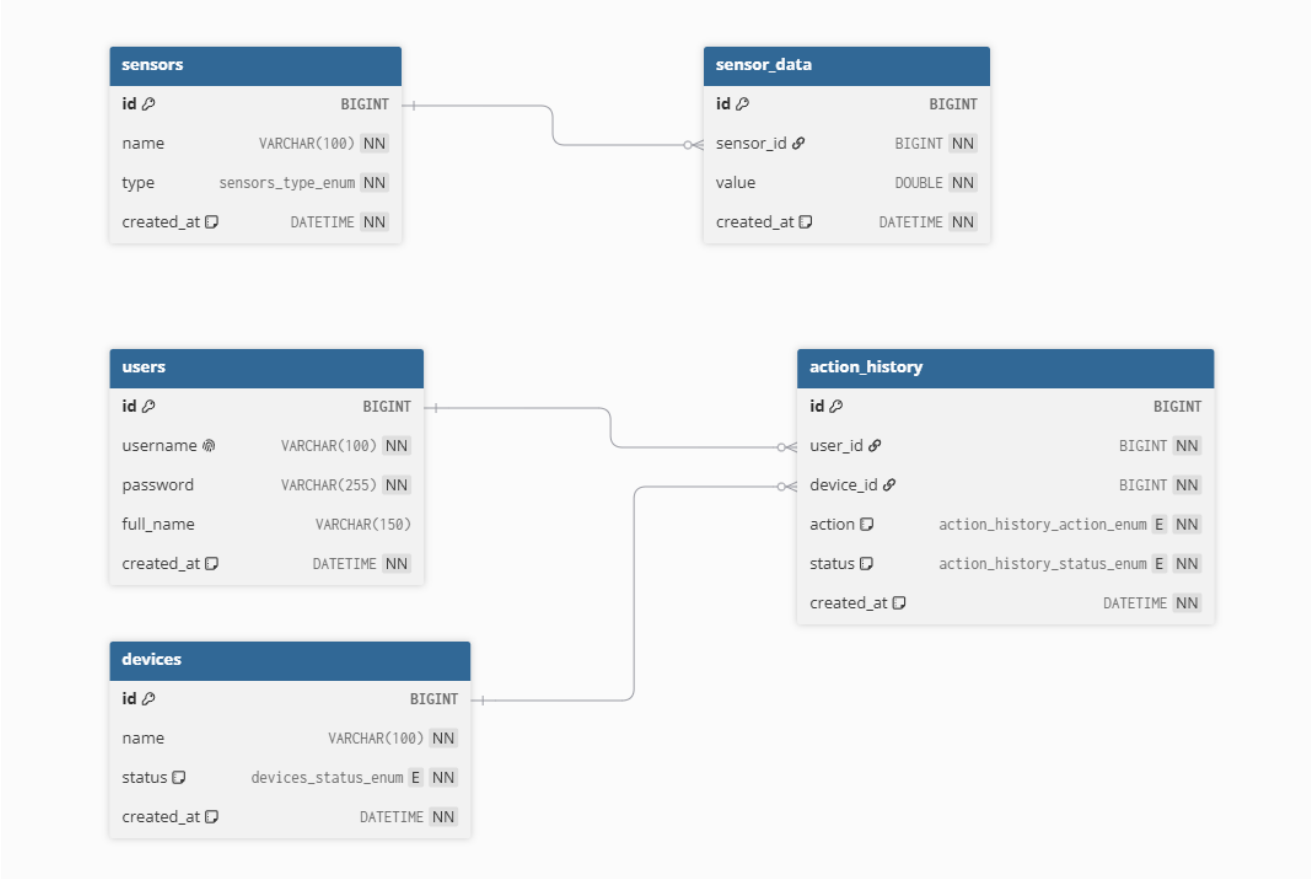
Cơ sở dữ liệu gồm năm bảng chính:

- users
- sensors
- devices
- sensor_data
- action_history

Quan hệ:

- Một Sensor có nhiều SensorData.

- Một User có thể thực hiện nhiều ActionHistory.
- Một Device có nhiều ActionHistory.



Hình 2.2. Sơ đồ ERD của hệ thống

2.2.2. Bảng users

Lưu thông tin tài khoản người dùng.

Field	Kiểu dữ liệu	Mô tả	Khóa/ Ràng buộc
id	BIGINT	Khóa chính	PK, AUTO_INCREMENT
username	VARCHAR	Tên đăng nhập	UNIQUE, NOT NULL
password	VARCHAR	Mật khẩu đã mã hóa	NOT NULL
full_name	VARCHAR	Họ tên	NOT NULL
created_at	DATETIME	Thời gian tạo	NOT NULL

2.2.3. Bảng sensors

Lưu danh sách cảm biến.

Field	Kiểu dữ liệu	Mô tả	Khóa/ Ràng buộc
id	BIGINT	Khóa chính	PK, AUTO_INCREMENT
name	VARCHAR	Tên cảm biến	NOT NULL
type	VARCHAR	Humidity/Temperature/Light	NOT NULL
created_at	DATETIME	Thời gian tạo	NOT NULL

2.2.4. Bảng devices

Lưu danh sách thiết bị có thể điều khiển.

Field	Kiểu dữ liệu	Mô tả	Khóa/ Ràng buộc
id	BIGINT	Khóa chính	PK, AUTO_INCREMENT
name	VARCHAR	Tên thiết bị	NOT NULL
status	VARCHAR	Trạng thái hiện tại	NOT NULL
created_at	DATETIME	Thời gian tạo	NOT NULL

2.2.5. Bảng sensor_data

Lưu dữ liệu được thu thập từ cảm biến.

Field	Kiểu dữ liệu	Mô tả	Khóa/ Ràng buộc
id	BIGINT	Khóa chính	PK, AUTO_INCREMENT

Field	Kiểu dữ liệu	Mô tả	Khóa/ Ràng buộc
sensor_id	BIGINT	Khóa ngoại đến sensors	NOT NULL
value	DOUBLE	Giá trị đo	NOT NULL
created_at	DATETIME	Thời gian lưu	NOT NULL

2.2.6. Bảng action_history

Lưu lịch sử điều khiển thiết bị.

Field	Kiểu dữ liệu	Mô tả	Khóa/Ràng buộc
id	BIGINT	Khóa chính	PK, AUTO_INCREMENT
user_id	BIGINT	Người thực hiện	NOT NULL
device_id	BIGINT	Thiết bị	NOT NULL
action	VARCHAR	ON/OFF	NOT NULL
status	VARCHAR	LOADING/ON/OFF/ FAILED	NOT NULL
created_at	DATETIME	Thời gian thao tác	NOT NULL

III. THIẾT KẾ CHI TIẾT

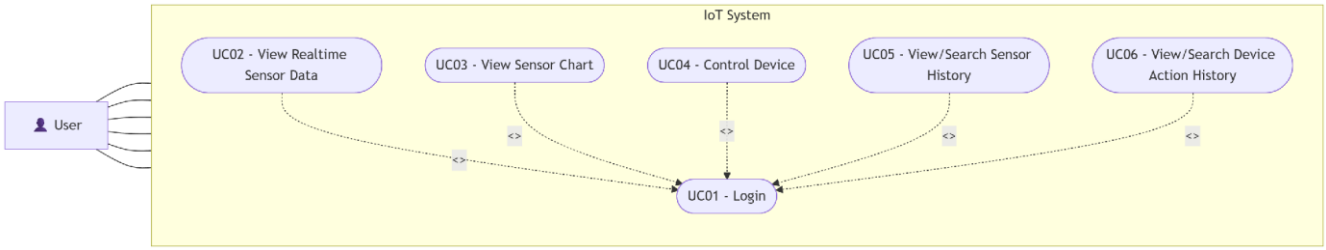
3.1. Tổng quan use case

3.1.1. Actor

Actor chính của hệ thống: User

User là người sử dụng Web để theo dõi dữ liệu cảm biến và điều khiển các thiết bị.

3.1.2. Danh sách Use Case



Hình 3.2. Sơ đồ Use Case tổng quan của hệ thống

ID	Use Case	Mô tả
UC01	Login	Đăng nhập vào hệ thống
UC02	View Realtime Sensor Data	Xem dữ liệu cảm biến thời gian thực
UC03	View Sensor Chart	Xem biểu đồ dữ liệu cảm biến
UC04	Control Device	Bật/tắt thiết bị
UC05	View/Search Sensor History	Xem và tìm kiếm lịch sử cảm biến
UC06	View/Search Device Action History	Xem và tìm kiếm lịch sử điều khiển

3.2 Đặc tả chi tiết use case

3.2.1. UC01 – Login

a. Đặc tả use case

Thuộc tính	Nội dung
Actor	User
Precondition	User có tài khoản hợp lệ
Trigger	User truy cập trang Login, nhập username, password và nhấn login

Thuộc tính	Nội dung
Postcondition	User đăng nhập thành công và được chuyển đến Dashboard

- Luồng thực hiện:
  1. User nhập username, password và nhấn login.
  2. Frontend gọi API: POST api/auth/login và gửi kèm username, password đến Backend.
  3. Backend yêu cầu lấy dữ liệu từ bảng User của Database theo username.
  4. Database trả về kết quả bảng user cho Backend.
  5. Backend kiểm tra thông tin đăng nhập.
  6. Backend tạo và trả token cho Frontend .
  7. Frontend hiển thị trang Dashboard cho User

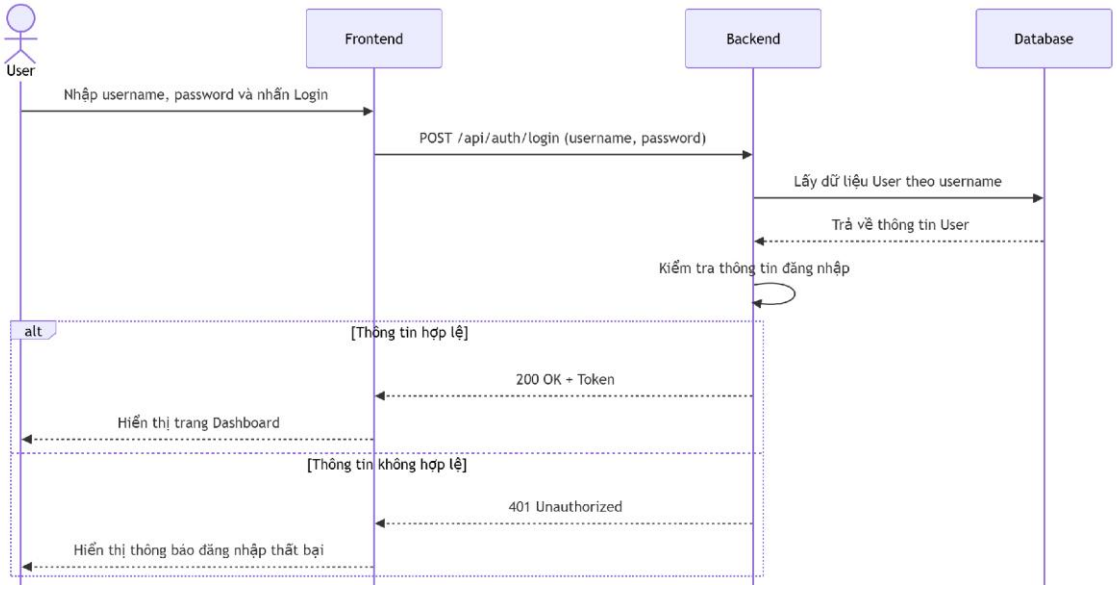
- Luồng ngoại lệ

Thiếu dữ liệu: frontend yêu cầu nhập đầy đủ

Nếu username hoặc password không hợp lệ:

1. Backend trả kết quả đăng nhập thất bại cho Frontend.
2. Frontend hiển thị thông báo lỗi cho User.
3. User nhập lại thông tin.

b. Sequence diagram



Hình 3.3. Sequence Diagram UC01 – Login

c. API Docs

- Method: POST
- Endpoint: /api/auth/login
- Request body:

```
{
 "username":"",
 "password":""
}
```

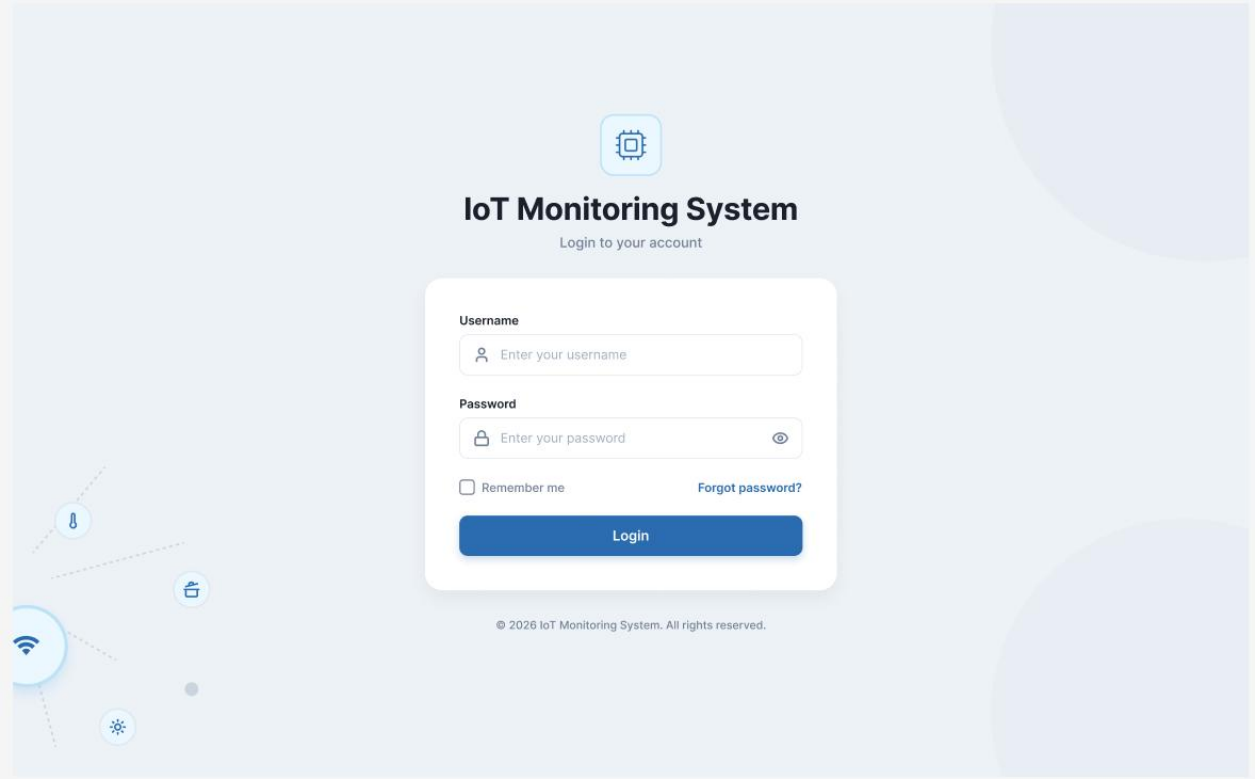
Response:

```
{
 "token":"access-token",
 "user":
 {
  "id":,
  "username":"",
  "fullName":""
 }
}
```

d. Mapping dữ liệu đăng nhập

Request data	Database	Field
username	users	username
password	users	password

e. Hình ảnh giao diện



Hình 3.4. Giao diện màn hình đăng nhập

3.2.2. UC02 – View Realtime Sensor Data

a. Đặc tả use case

Thuộc tính	Nội dung
Actor	User
Precondition	User đã đăng nhập
Trigger	User truy cập Dashboard
Postcondition	Dashboard hiển thị dữ liệu gần nhất và tự động cập nhật khi có bản ghi mới.

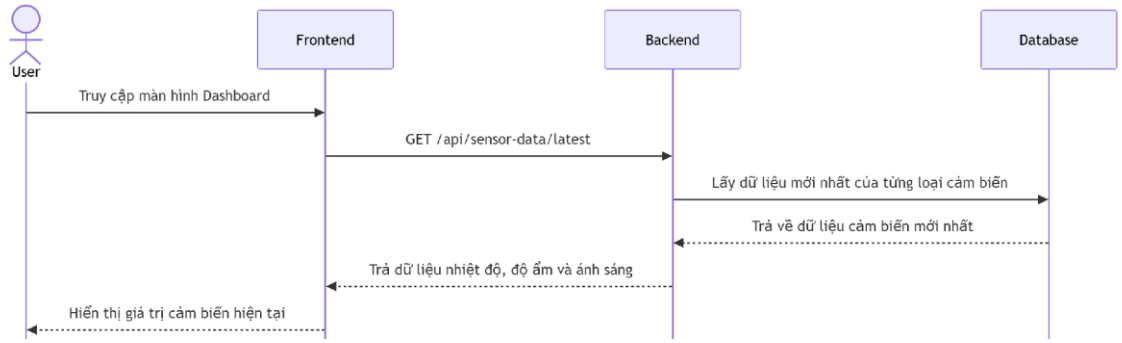
b. Luồng khi User vừa mở trang Dashboard

b.1. Luồng thực hiện

1. User truy cập màn hình Dashboard.
2. Frontend gọi API: GET /api/sensor-data/latest lấy giá trị cảm biến mới nhất

3. Backend lấy dữ liệu mới nhất của từng loại cảm biến từ bảng sensor_data trong Database.
4. Backend trả dữ liệu về Frontend.
5. Frontend hiển thị giá trị nhiệt độ, độ ẩm và ánh sáng hiện tại cho User.

b.2. Sequence diagram



Hình 3.5. Sequence Diagram UC02 – View Realtime Sensor Data khi mở Dashboard

b.3. API docs

- Method: GET
- Endpoint: /api/sensor-data/latest
- Response:

```
{
  "temperature":,
  "humidity":,
  "light":,
  "createdAt":
}
```

b.4. Mapping dữ liệu response

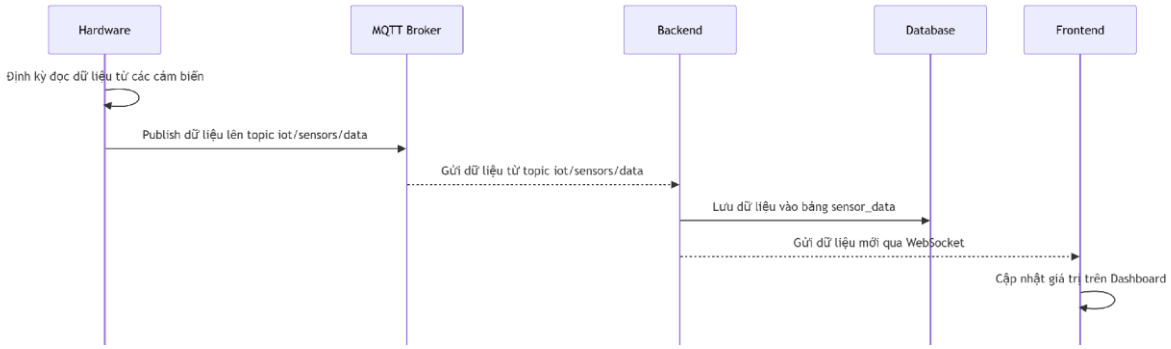
Response data	Database	Field
temperature	sensor_data	value
humidity	sensor_data	value
light	sensor_data	value
createdAt	sensor_data	created_at

c. Luồng cảm biến đo lại giá trị sau mỗi khoảng thời gian

c.1. Luồng thực hiện

1. Hardware định kỳ đọc dữ liệu từ các cảm biến.
2. Hardware publish dữ liệu mới lên MQTT Broker qua topic: iot/sensors/data
3. Backend subscribe topic iot/sensors/data, nhận dữ liệu và lưu vào bảng sensor_data.
4. Backend gửi dữ liệu mới đến Frontend thông qua WebSocket.
5. Frontend nhận dữ liệu và cập nhật các giá trị trên Dashboard.

c.2. Sequence diagram



Hình 3.6. Sequence Diagram UC02 – Cập nhật dữ liệu cảm biến thời gian thực

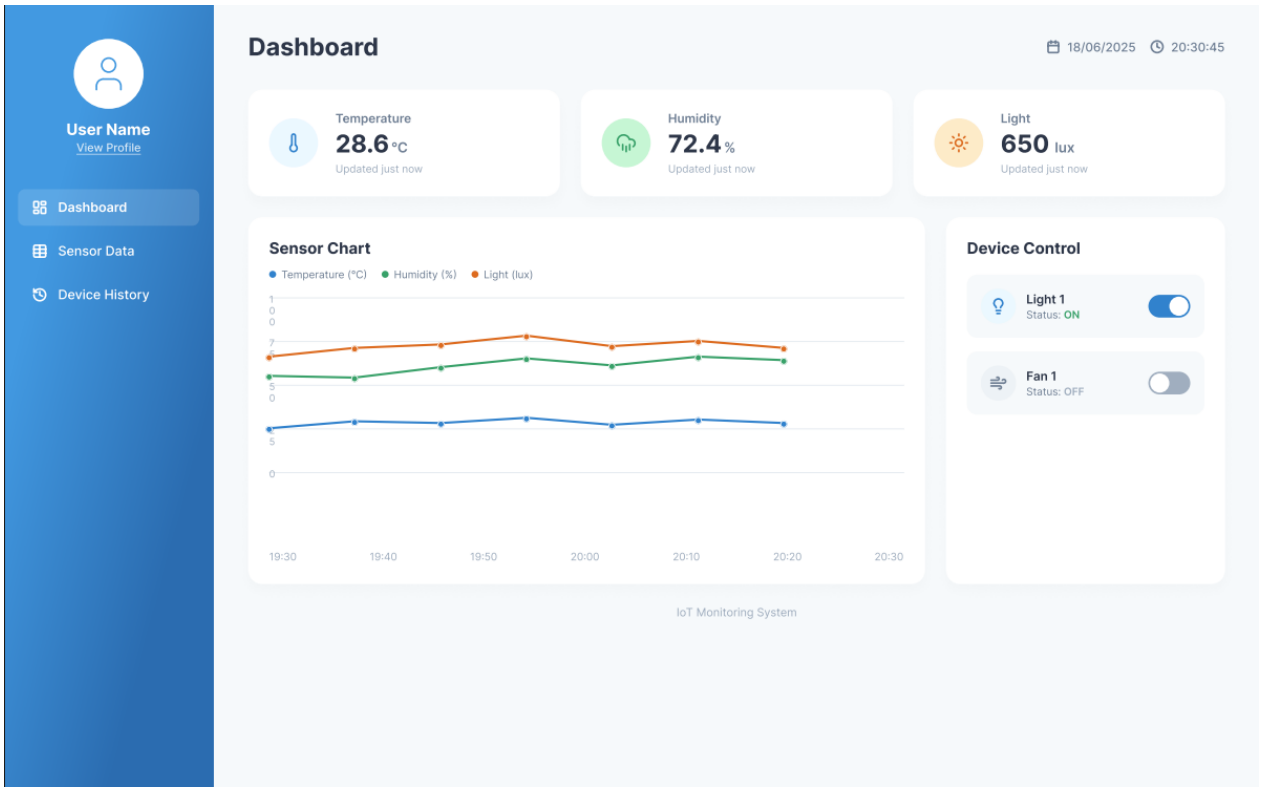
c.3. Data truyền qua socket

```
{
 "temperature":,
 "humidity":,
 "light":,
 "createdAt":
}
```

c.4. Mapping dữ liệu truyền qua socket:

Socket data	Database	Field
temperature	sensor_data	value
humidity	sensor_data	value
light	sensor_data	value
createdAt	sensor_data	created_at

d. Hình ảnh giao diện



Hình 3.7. Giao diện màn hình Dashboard

3.2.3. UC03 – View Sensor Chart

a. Đặc tả use case

Thuộc tính	Nội dung
Actor	User
Precondition	User đã đăng nhập, hệ thống đã có dữ liệu cảm biến
Trigger	User mở Dashboard
Postcondition	Biểu đồ dữ liệu được hiển thị

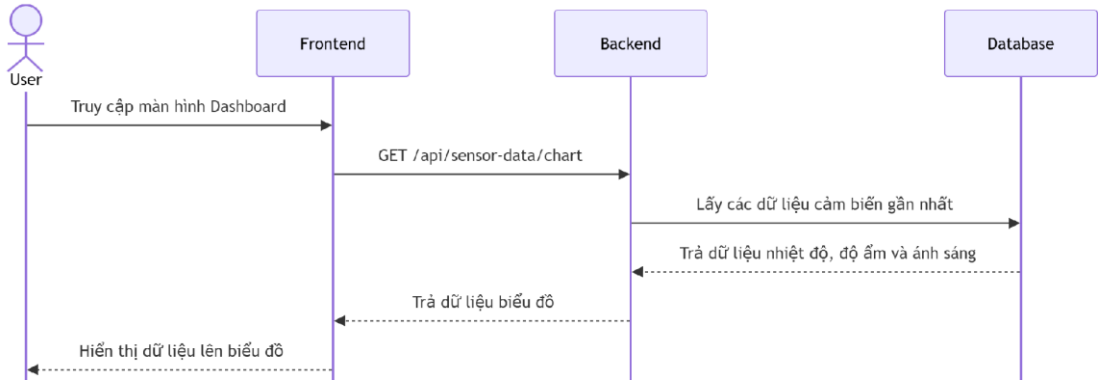
b. Luồng khi User vừa mở trang Dashboard

b.1 Luồng thực hiện

1. User truy cập màn hình Dashboard.
2. Frontend gọi API: GET /api/sensor-data/chart để lấy 15 giá trị gần nhất của từng loại cảm biến.

3. Backend lấy các dữ liệu gần nhất của cảm biến nhiệt độ, độ ẩm và ánh sáng từ bảng sensor_data trong Database.
4. Backend trả dữ liệu về Frontend.
5. Frontend nhận dữ liệu và hiển thị lên biểu đồ cho User.

b.2 Sequence diagram



Hình 3.8. Sequence Diagram UC03 – View Sensor Chart khi mở Dashboard

b.3 API docs

- Method: GET
- Endpoint: /api/sensor-data/chart
- Response:

{

"temperature": [

{ "time": " ", "value": },

{ "time": " ", "value": },

….

],

"humidity": [

{ "time": " ", "value": },

{ "time": " ", "value": },

…

],

"light": [

{ "time": " ", "value": },

{ "time": " ", "value": },

…

]

}

b.4 Mapping dữ liệu response

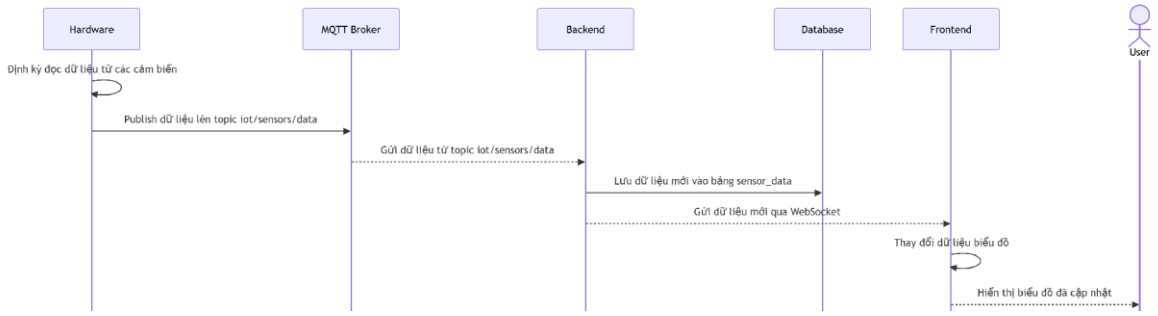
Response data	Database	Field
temperature[].value	sensor_data	value
temperature[].time	sensor_data	created_at
humidity[].value	sensor_data	value
humidity[].time	sensor_data	created_at
light[].value	sensor_data	value
light[].time	sensor_data	created_at

c. Luồng khi cảm biến đo được dữ liệu mới

c.1 Luồng thực hiện

1. Hardware định kỳ đọc dữ liệu từ các cảm biến.
2. Hardware publish dữ liệu mới lên MQTT Broker qua topic: iot/sensors/data
3. Backend subscribe topic iot/sensors/data, nhận dữ liệu mới và lưu vào bảng sensor_data.
4. Backend gửi dữ liệu mới đến Frontend thông qua WebSocket.
5. Frontend nhận dữ liệu mới, thay đổi dữ liệu biểu đồ và hiển thị user.

c.2 Sequence diagram



Hình 3.9. Sequence Diagram UC03 – Cập nhật biểu đồ khi có dữ liệu mới

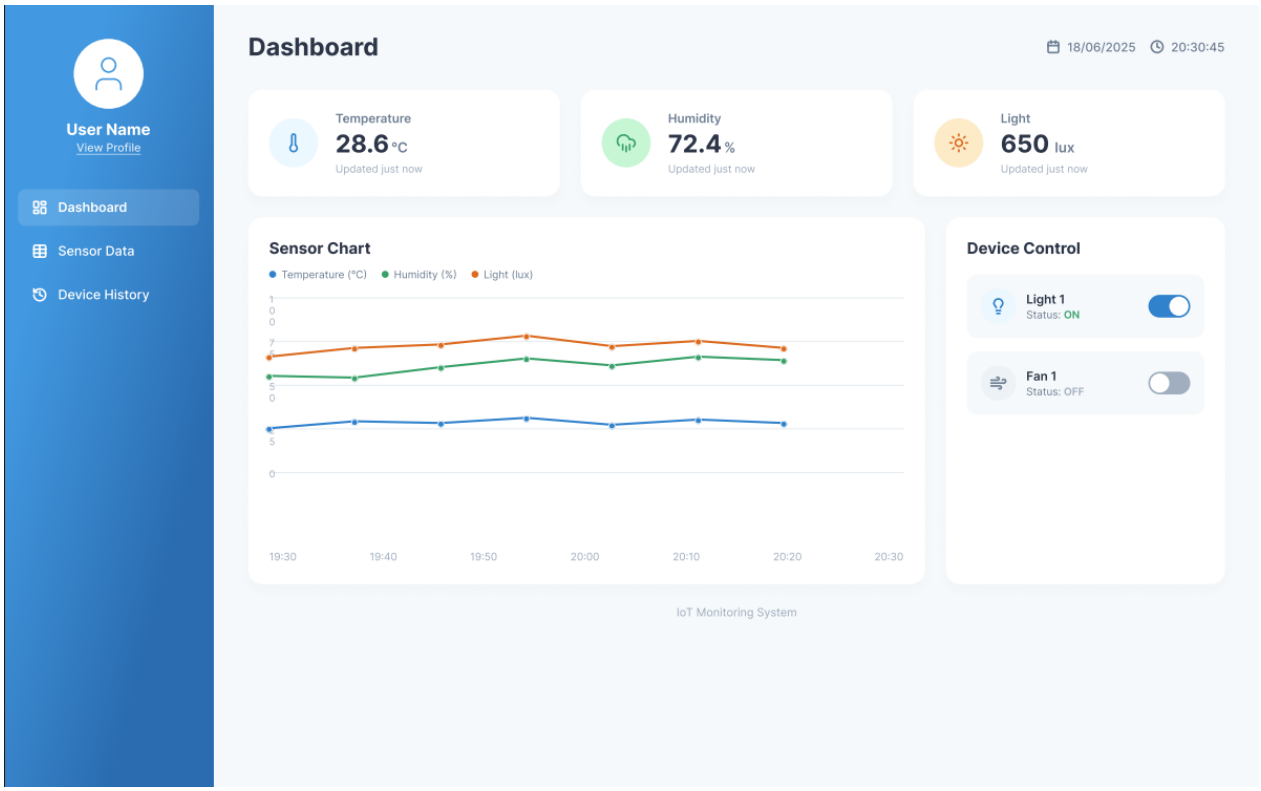
c.3 Dữ liệu WebSocket:

```
{
    "temperature": ,
    "humidity": ,
    "light": ,
    "createdAt":
  }
```

c.4 Mapping dữ liệu truyền qua socket:

Socket data	Database	Field
temperature	sensor_data	value
humidity	sensor_data	value
light	sensor_data	value
createdAt	sensor_data	created_at

d. Hình ảnh giao diện



Hình 3.10. Giao diện màn hình Dashboard

3.2.4. UC04 – Control Device

a. Đặc tả use case

Thuộc tính	Nội dung
Actor	User
Precondition	User đã đăng nhập
Trigger	User nhấn nút ON/OFF
Postcondition	Thiết bị được bật/tắt và trạng thái được lưu

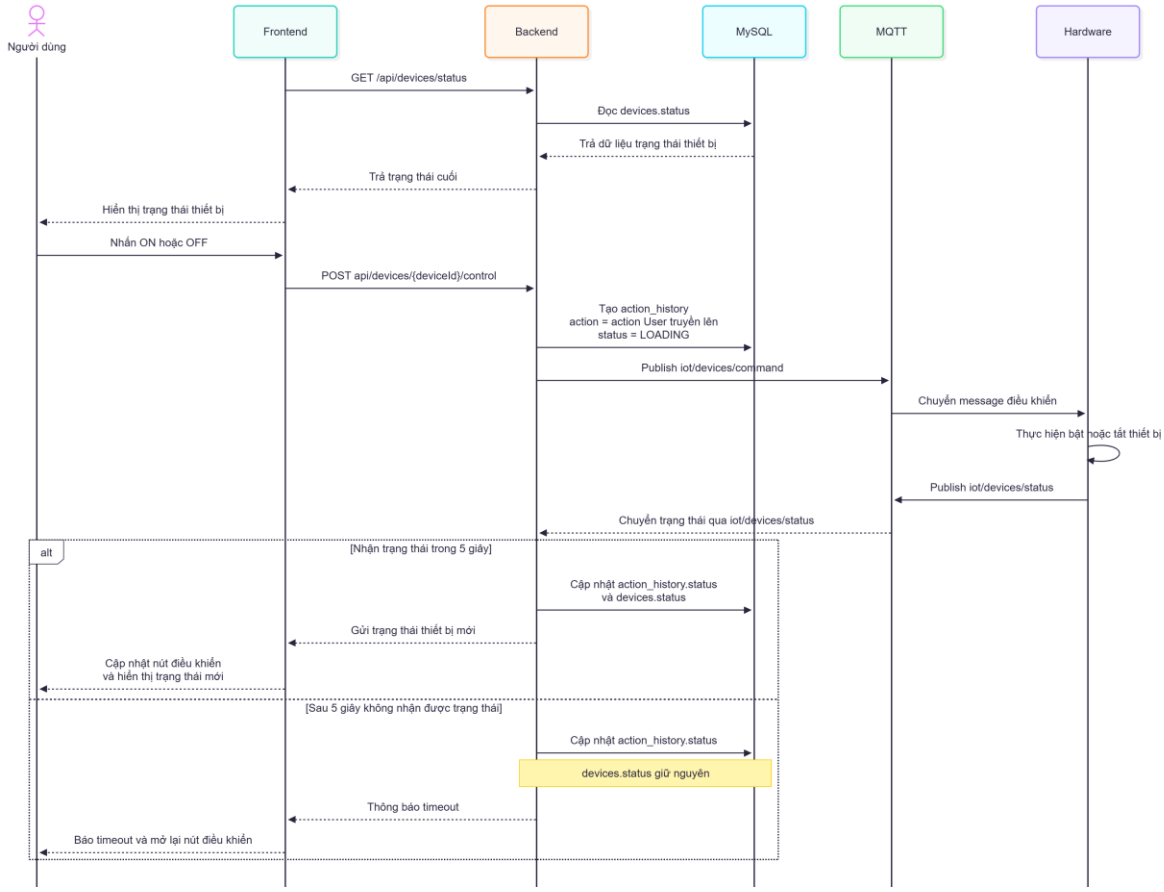
- Luồng thực hiện:

1. Frontend gọi GET /api/devices/status.
2. Backend yêu cầu đọc devices.status từ database.
3. Database trả dữ liệu cho Backend.
4. Backend trả dữ liệu trạng thái cuối cho Frontend.

5. Frontend hiển thị trạng thái thiết bị cho User.
6. User nhấn ON hoặc OFF trên Dashboard.
7. Frontend gửi API: POST api/devices/{deviceId}/control đến Backend.
8. Backend tạo một bản ghi trong action_history của Database: có action là action user truyền lên, và status hiện tại Loading
9. Backend publish lên topic: iot/devices/command lệnh điều khiển đến MQTT Broker.
10. MQTT Broker chuyển message đến Hardware.
11. Hardware thực hiện bật hoặc tắt thiết bị.
12. Hardware gửi trạng thái thực tế lên MQTT Broker qua topic: iot/devices/status.
13. Backend nhận trạng thái phản hồi từ MQTT Broker qua topic: iot/devices/status.
14. Backend cập nhật bảng action_history.status và devices.status.
15. Frontend nhận dữ liệu từ Backend và cập nhật nút điều khiển.
16. Frontend hiển thị dữ liệu mới cho user.

- Luồng ngoại lệ: Không lấy được trạng thái cuối: khóa nút điều khiển và yêu cầu tải lại. Thiết bị không tồn tại hoặc action sai: API từ chối. Sau 5 giây không nhận được topic trạng thái: action_history chuyển FAILED, devices.status giữ nguyên; giao diện báo timeout và mở lại nút điều khiển.

b. Sequence diagram



Hình 3.11. Sequence Diagram UC04 – Control Device

c. API docs

API lấy trạng thái cuối của các thiết bị:

- Method: GET
- Endpoint: /api/devices/status
- Response:

```
[
  {
   "deviceId": ,
   "status": ""
  }
]
```

API điều khiển thiết bị:

- Method: POST
- Endpoint: /api/devices/{deviceId}/control
- Request body:

```
{
  "action":""
}
```

- Response:

```
{
  "deviceId":,
  "action":"",
  "status":""
}
```

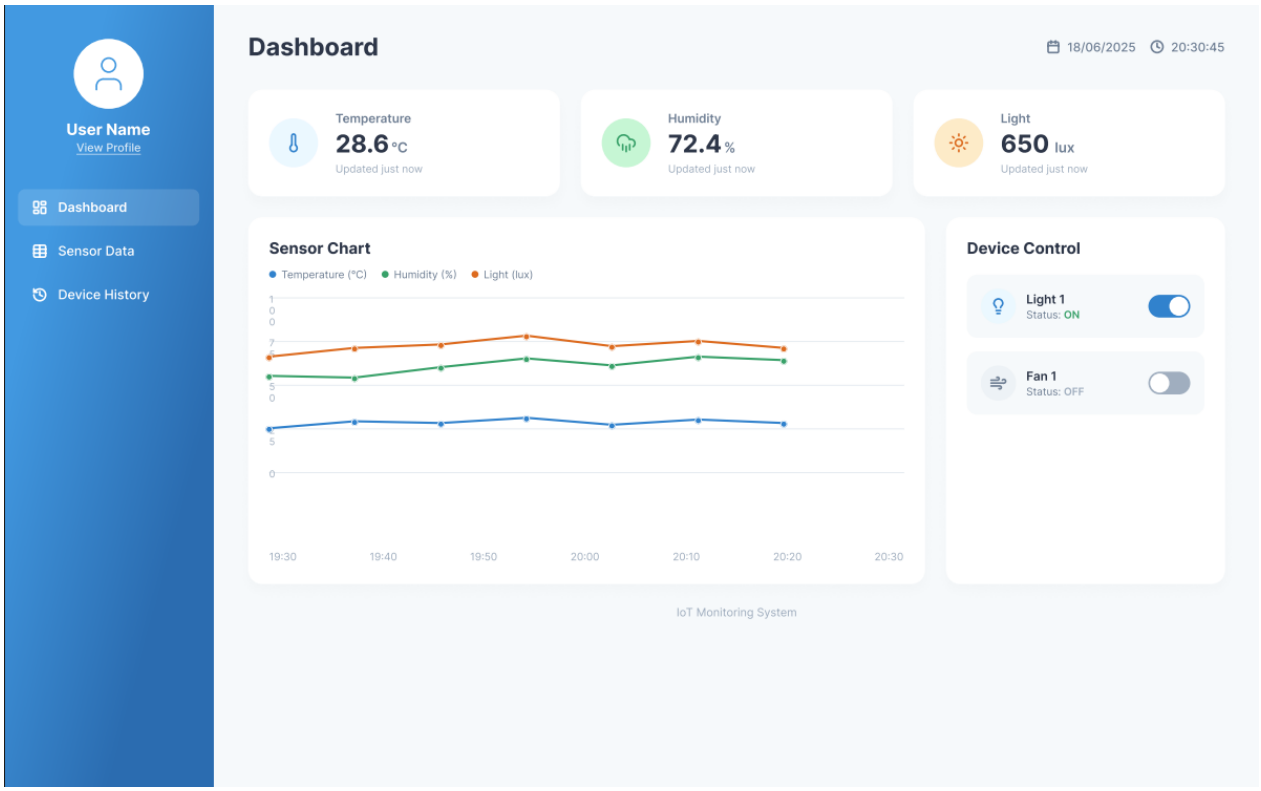
- Sau khi Hardware phản hồi, trạng thái được gửi qua Websocket:

```
{
  "deviceId":,
  "status":""
}
```

d. Mapping dữ liệu

Data	Database	Field
deviceId	action_history	device_id
action	action_history	action
status	action_history	status

e. Hình ảnh giao diện



Hình 3.12. Giao diện màn hình Dashboard

3.2.5. UC05 – View/Search Sensor History

a. Đặc tả use case

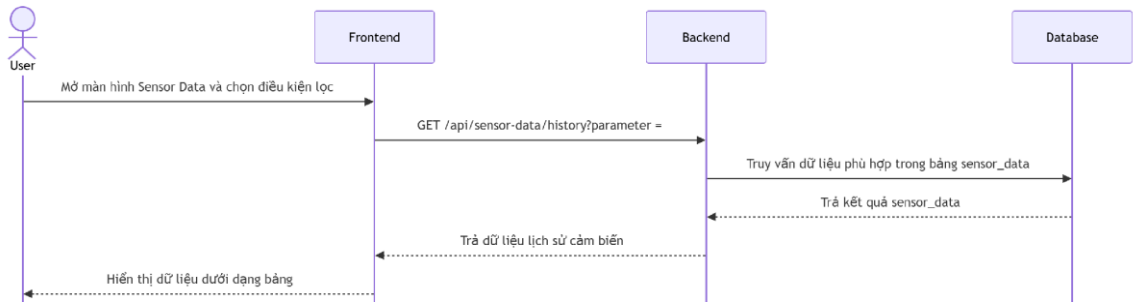
Thuộc tính	Nội dung
Actor	User
Precondition	User đã đăng nhập
Trigger	User mở Sensor Data
Postcondition	Danh sách dữ liệu phù hợp được hiển thị

- Luồng thực hiện:

1. User mở màn hình Sensor Data và chọn điều kiện lọc là cảm biến/thời gian
2. Frontend gửi API: GET /api/sensor-data/history?parameter = lấy lịch sử cảm biến.
3. Backend truy vấn các dữ liệu phù hợp trong bảng sensor_data.
4. Database trả kết quả bảng sensor_data cho Backend.

5. Backend trả kết quả lại cho Frontend
6. Frontend hiển thị dữ liệu thành bảng cho User.

b. Sequence diagram



Hình 3.13. Sequence Diagram UC05 – View/Search Sensor History

c. API docs

- Method: GET
- Endpoint: /api/sensor-data/history
- Query parameter:

searchType
value
page
size

sortBy

sortDirection

- Response:

```
{
 "content":[
{
    "id":,
    "sensorType":"",
    "value":,
    "createdAt":""
},
{
    "id":,
    "sensorType":"",
    "value":,
```

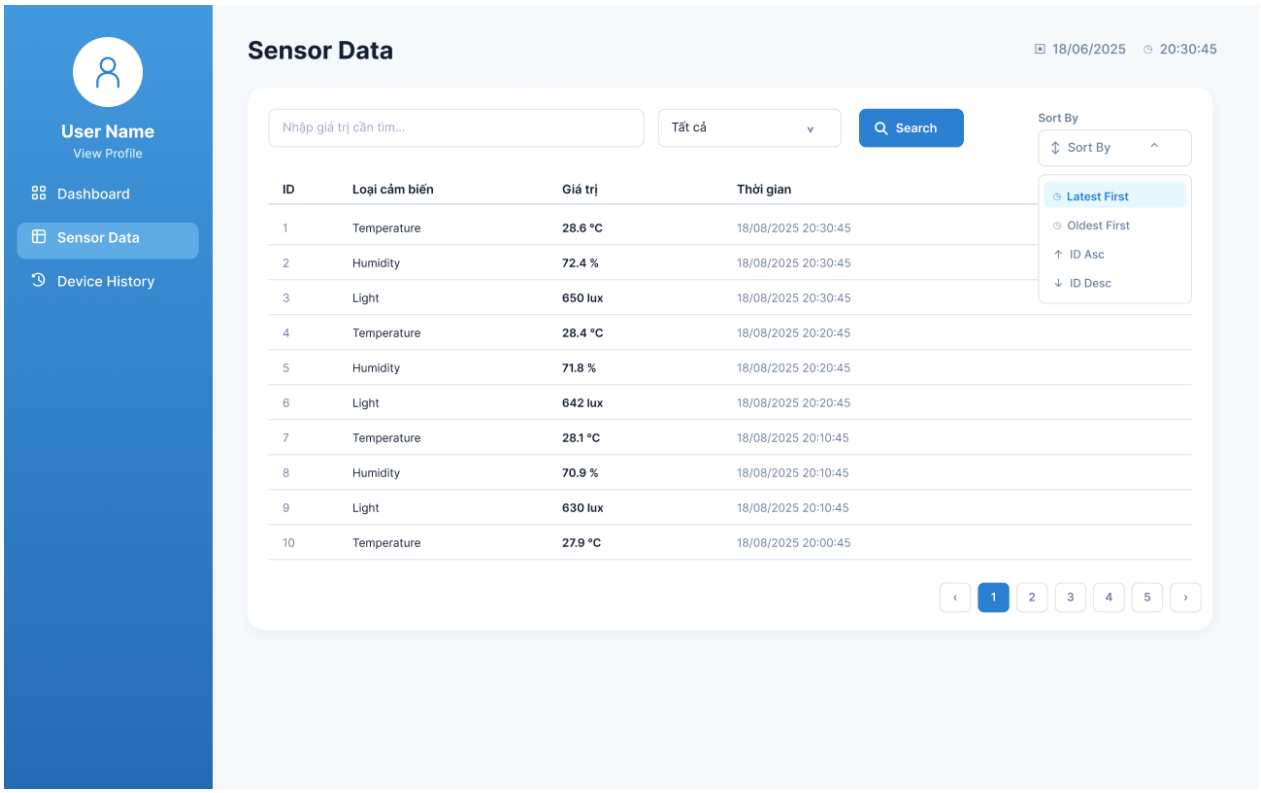
"createdAt":""
},
…
],
"page": ,
"size": ,
"totalElements": ,
"totalPages": ,
"first": ,
"last": ,
"numberOfElements": ,
"empty":

}

d. Mapping dữ liệu

Data	Database	Field
id	sensor_data	id
sensorType	sensors	type
value	sensor_data	value
createdAt	sensor_data	created_at

e. Hình ảnh giao diện



Hình 3.14. Giao diện màn hình Sensor Data

3.2.6. UC06 – View/Search Device Action History

a. Đặc tả use case

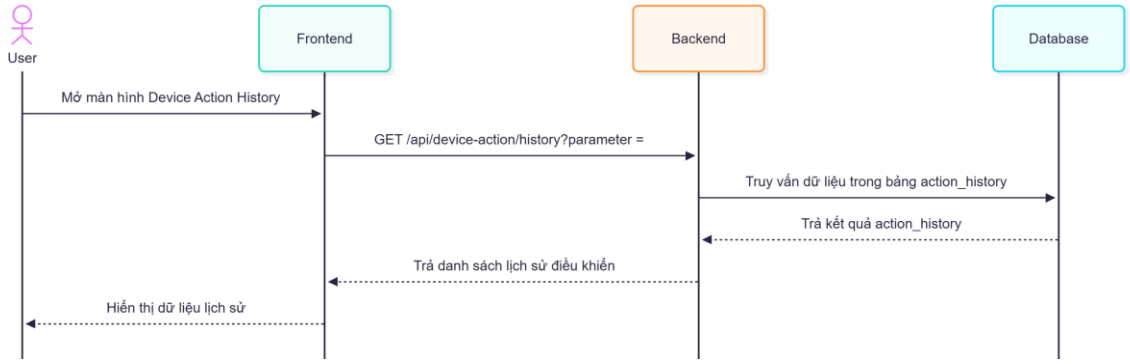
Thuộc tính	Nội dung
Actor	User
Precondition	User đã đăng nhập
Trigger	User mở Device Action History
Postcondition	Lịch sử điều khiển được hiển thị

- Luồng chính

1. User mở màn hình Device Action History.
2. Frontend gửi API: GET /api/device-action/history?parameter = đến Backend.
3. Backend truy vấn dữ liệu trong bảng action_history.

4. Database trả kết quả cho Backend.
5. Backend trả danh sách lịch sử cho Frontend.
6. Frontend hiển thị dữ liệu cho User.

b. Sequence diagram



Hình 3.15. Sequence Diagram UC06 – View/Search Device Action History

c. API docs

- Method: GET
- Endpoint: /api/device-action/history
- Query parameters:

deviceId
action
status
time
page
size

sortBy

sortDirection

- Response:

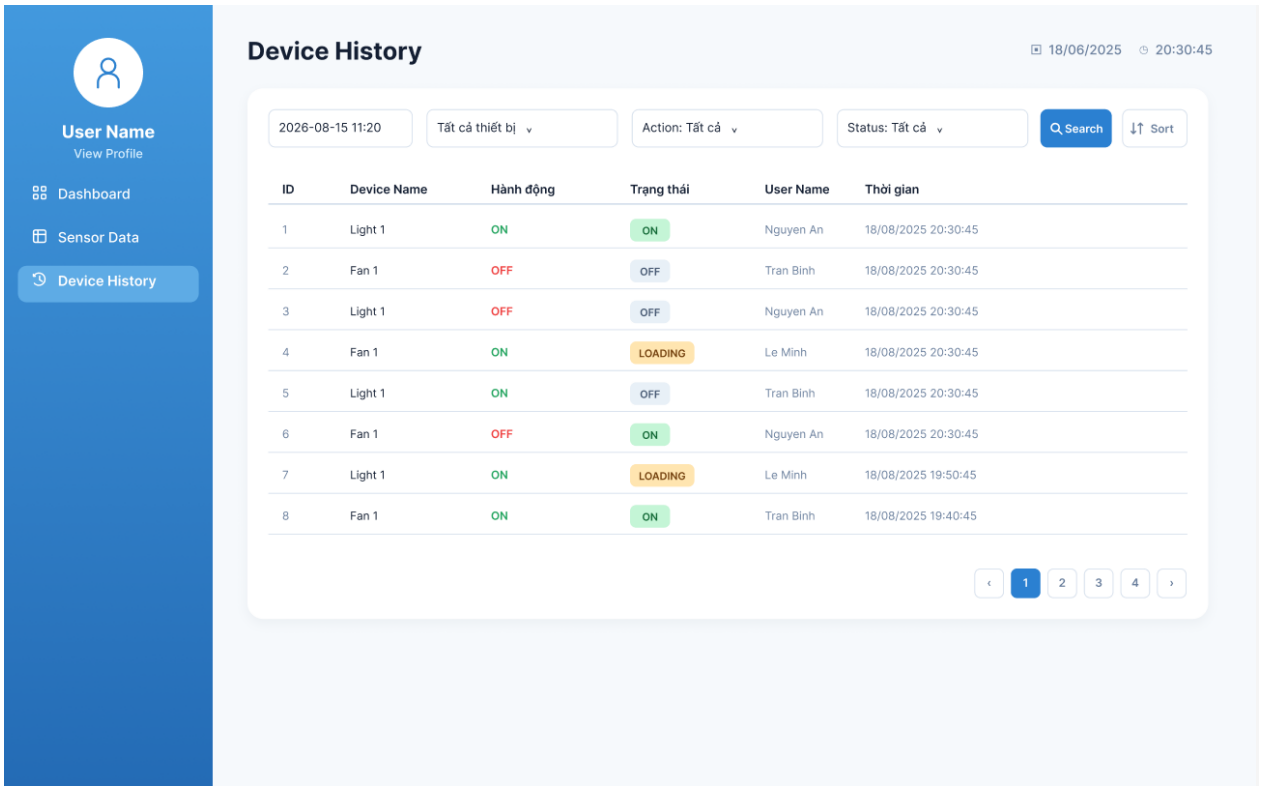
```
{
 "content":[
   {
    "id":,
    "userName":"",
    "deviceName":"",
    "action":"",
    "status":"",
```

```
      "createdAt":""
    },
…
  ],
"page": ,
  "size": ,
  "totalElements": ,
  "totalPages": ,
  "first": ,
  "last": ,
  "numberOfElements": ,
  "empty":
}
```


d. Mapping dữ liệu

Data	Database	Field
id	action_history	id
userName	users	full_name
deviceName	devices	name
action	action_history	action
status	action_history	status
createdAt	action_history	created_at

e. Hình ảnh giao diện



Hình 3.16. Giao diện màn hình Device Action History

IV. KẾT QUẢ THỰC HIỆN

4.1. Kết quả triển khai hệ thống

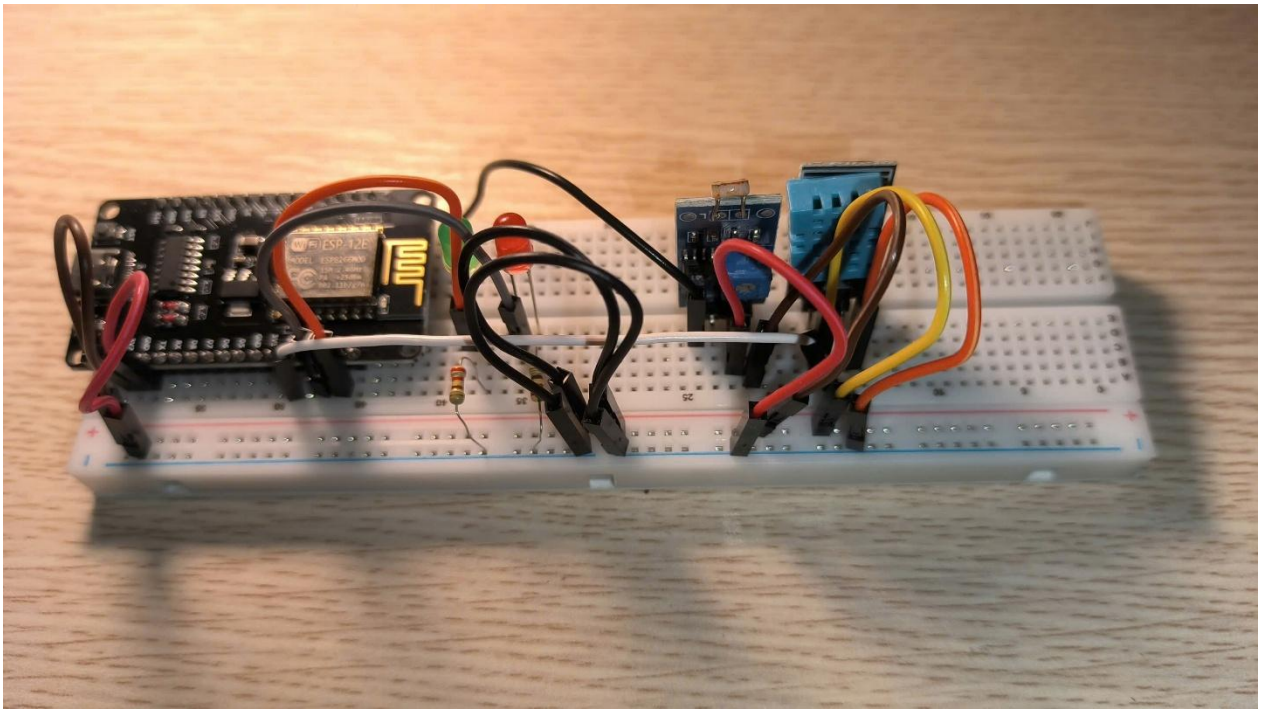
Hệ thống giám sát môi trường và điều khiển thiết bị được tổ chức theo kiến trúc gồm năm thành phần: giao diện Web, Backend Spring Boot, cơ sở dữ liệu MySQL, MQTT Broker và phần cứng ESP8266. Mỗi thành phần đảm nhiệm một nhóm nhiệm vụ riêng, đồng thời phối hợp để truyền dữ liệu đo từ cảm biến đến người dùng và chuyển lệnh điều khiển theo chiều ngược lại.

Frontend sử dụng HTML, CSS và JavaScript để hiển thị thông tin; Backend cung cấp REST API, xử lý nghiệp vụ và trao đổi bản tin MQTT; MySQL lưu thông tin người dùng, dữ liệu cảm biến và lịch sử thao tác. ESP8266 là nút kết nối cảm biến, tiếp nhận lệnh bật/tắt hai đèn và gửi phản hồi trạng thái. WebSocket được sử dụng để chuyển dữ liệu mới từ Backend đến Dashboard mà không phải tải lại trang.

a. Triển khai phần cứng và chương trình trên ESP8266

Khối phần cứng sử dụng ESP8266 làm bộ điều khiển trung tâm, kết nối với các cảm biến nhiệt độ, độ ẩm, ánh sáng và hai đèn. Chương trình trên ESP8266 thực hiện kết nối Wi-Fi, thiết lập phiên MQTT và đọc dữ liệu cảm biến theo chu kỳ. Các giá trị đo được đóng gói thành thông điệp và gửi lên topic iot/sensors/data để Backend tiếp nhận, lưu trữ và chuyển đến giao diện giám sát.

Đối với chức năng bật/tắt đèn, ESP8266 lắng nghe các yêu cầu từ topic iot/devices/command. Sau khi xử lý lệnh ON hoặc OFF cho thiết bị tương ứng, bộ điều khiển gửi trạng thái thực tế qua topic iot/devices/status. Phản hồi từ phần cứng là căn cứ để cập nhật trạng thái thiết bị và hoàn tất lịch sử thao tác, tránh xác nhận thành công ngay khi người dùng vừa nhấn nút.



Hình 4.1. Mô hình ESP8266 kết nối cảm biến và hai đèn

b. Triển khai MQTT Broker

MQTT Broker Mosquitto đảm nhiệm việc trung chuyển bản tin giữa ESP8266 và Backend theo cơ chế Publish/Subscribe. Luồng dữ liệu được tách thành ba topic: iot/sensors/data truyền số liệu cảm biến, iot/devices/command truyền yêu cầu điều khiển và iot/devices/status truyền trạng thái sau khi thiết bị thực hiện lệnh. Việc phân chia topic giúp Backend xử lý độc lập dữ liệu đo và phản hồi điều khiển, đồng thời thuận tiện kiểm tra quá trình gửi và nhận thông điệp.

c. Khởi tạo và tổ chức cơ sở dữ liệu MySQL

Cơ sở dữ liệu được xây dựng theo sơ đồ ERD ở phần II với năm bảng: users, sensors, devices, sensor_data và action_history. Bảng users chứa thông tin xác thực; sensors và devices quản lý danh mục cảm biến, thiết bị; sensor_data lưu các giá trị đo kèm thời điểm ghi nhận; action_history lưu người thao tác, thiết bị được chọn, hành động ON/OFF và kết quả xử lý.

Các quan hệ một - nhiều giữa sensors và sensor_data, giữa users hoặc devices với action_history hỗ trợ truy vấn dữ liệu theo đúng đối tượng. Trường created_at tại hai bảng lịch sử phục vụ sắp xếp theo thời gian, tìm kiếm và phân trang trên giao diện. Trạng thái hiện tại của đèn được quản lý riêng trong devices, không đồng nhất với lệnh ON/OFF mà người dùng gửi lên.

d. Triển khai Backend bằng Java Spring Boot

Backend được xây dựng trên Java Spring Boot, đóng vai trò trung tâm kết nối giao diện, MySQL và MQTT Broker. Hệ thống cung cấp các REST API cho đăng nhập, lấy số liệu cảm biến mới nhất, truy vấn dữ liệu biểu đồ, đọc trạng thái thiết bị, gửi lệnh điều khiển và xem hai loại lịch sử. Ngoài xử lý yêu cầu HTTP, Backend còn nhận dữ liệu từ topic iot/sensors/data, lưu vào sensor_data và đẩy thông tin mới đến trình duyệt thông qua WebSocket.

Trong quy trình điều khiển, Backend tiếp nhận yêu cầu POST /api/devices/{deviceId}/control, lưu một bản ghi action_history với trạng thái LOADING rồi gửi lệnh đến ESP8266 qua MQTT. Khi nhận được phản hồi, hệ thống cập nhật action_history.status và devices.status theo trạng thái ON hoặc OFF thực tế. Nếu hết thời gian chờ 5 giây mà không có xác nhận, bản ghi lịch sử chuyển FAILED, còn trạng thái cuối của thiết bị được giữ nguyên. Cơ chế này cho phép theo dõi cả lệnh thành công lẫn trường hợp mất phản hồi.

e. Triển khai giao diện Web bằng HTML, CSS và JavaScript

Giao diện Web gồm năm màn hình: Login, Dashboard, Sensor Data, Device Action History và Profile. HTML và CSS xây dựng bố cục hiển thị, còn JavaScript thực hiện gọi REST API, xử lý dữ liệu trả về và cập nhật nội dung trên trang. Khi người dùng mở Dashboard, dữ liệu ban đầu được lấy qua HTTP; các giá trị cảm biến và trạng thái đèn phát sinh sau đó được tiếp nhận qua WebSocket. Cách tổ chức này kết hợp được thao tác tra cứu theo yêu cầu với khả năng theo dõi thời gian thực.

Thành phần	Kết quả triển khai
Phần cứng ESP8266	Thu thập nhiệt độ, độ ẩm, ánh sáng; nhận lệnh ON/OFF cho Đèn 1 và Đèn 2, gửi trạng thái sau khi thực hiện.
MQTT Broker	Truyền dữ liệu cảm biến, lệnh điều khiển và phản hồi thiết bị trên ba topic độc lập.
MySQL	Tổ chức năm bảng users, sensors, devices, sensor_data, action_history để quản lý thông tin và lịch sử.
Backend Spring Boot	Cung cấp REST API, tương tác cơ sở dữ liệu, nhận/gửi MQTT và cập nhật Dashboard qua WebSocket.
Frontend Web	Tổ chức năm màn hình, phục vụ sáu use case và hiển thị dữ liệu theo thời gian thực.

4.2. Kết quả giao diện và chức năng

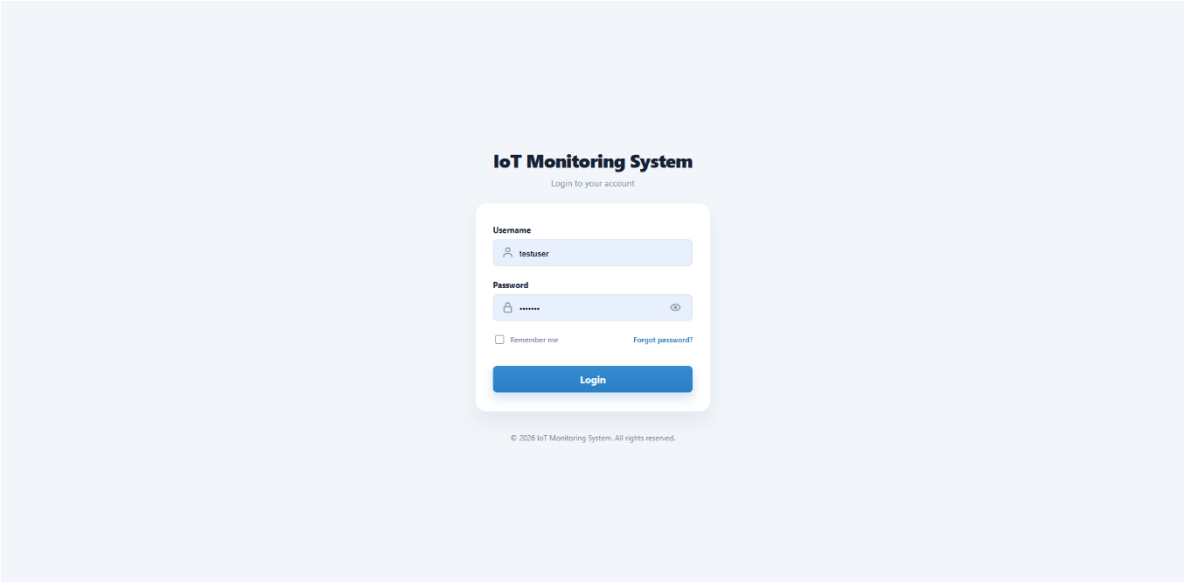
Các màn hình Web được bố trí theo nhóm thao tác của người dùng: xác thực tài khoản, giám sát môi trường, điều khiển đèn và tra cứu dữ liệu lịch sử. Dashboard

tập trung những chức năng cần cập nhật liên tục, trong khi Sensor Data và Device Action History cung cấp bảng dữ liệu có tìm kiếm, sắp xếp và phân trang. Các chức năng được liên kết thông qua REST API của Backend Spring Boot.

a. Màn hình đăng nhập

Màn hình Login gồm các trường username, password và nút đăng nhập. Sau khi người dùng nhập đủ thông tin, JavaScript gửi yêu cầu POST /api/auth/login để Backend đối chiếu với bảng users. Nếu xác thực thành công, phản hồi chứa token cùng thông tin người dùng và giao diện chuyển đến Dashboard. Trường hợp bỏ trống dữ liệu hoặc đăng nhập sai, màn hình hiển thị thông báo phù hợp và cho phép nhập lại mà không chuyển trang.

Chức năng chính: tiếp nhận tài khoản và mật khẩu; gửi yêu cầu xác thực; xử lý trường hợp không hợp lệ; chuyển đến Dashboard khi đăng nhập thành công.



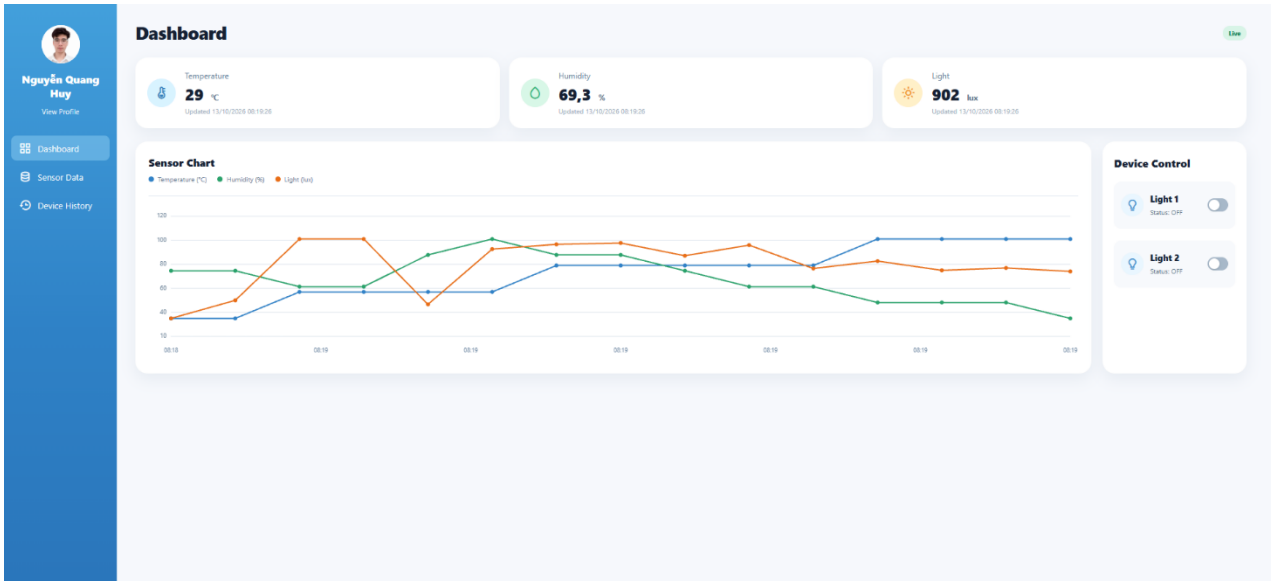
Hình 4.2. Kết quả giao diện đăng nhập

b. Dashboard - dữ liệu thời gian thực, biểu đồ và điều khiển đèn

Dashboard hiển thị đồng thời nhiệt độ, độ ẩm, ánh sáng và biểu đồ diễn biến theo thời gian. Khi trang được mở, Frontend gọi GET /api/sensor-data/latest để lấy số liệu mới nhất và GET /api/sensor-data/chart để nhận 15 giá trị gần nhất của từng loại cảm biến. Dữ liệu mới từ ESP8266 được Backend chuyển qua WebSocket, nhờ đó các thông số và điểm trên biểu đồ thay đổi ngay trên màn hình mà không cần thực hiện lại thao tác truy vấn.

Khu vực điều khiển hiển thị Đèn 1 và Đèn 2 cùng trạng thái ON/OFF tương ứng. Frontend lấy trạng thái ban đầu bằng GET /api/devices/status, sau đó gửi POST /api/devices/{deviceId}/control khi người dùng chọn lệnh. Trong thời gian chờ phản hồi, giao diện thể hiện trạng thái đang xử lý và hạn chế thao tác liên tiếp; khi Backend nhận được phản hồi MQTT, trạng thái hiển thị được cập nhật theo thiết bị. Nếu hết thời gian chờ, giao diện thông báo lỗi và giữ lại trạng thái đã xác nhận trước đó.

Chức năng chính: hiển thị các giá trị cảm biến hiện tại; biểu diễn số liệu thành biểu đồ; nhận cập nhật qua WebSocket; bật/tắt hai đèn; theo dõi kết quả xử lý lệnh.



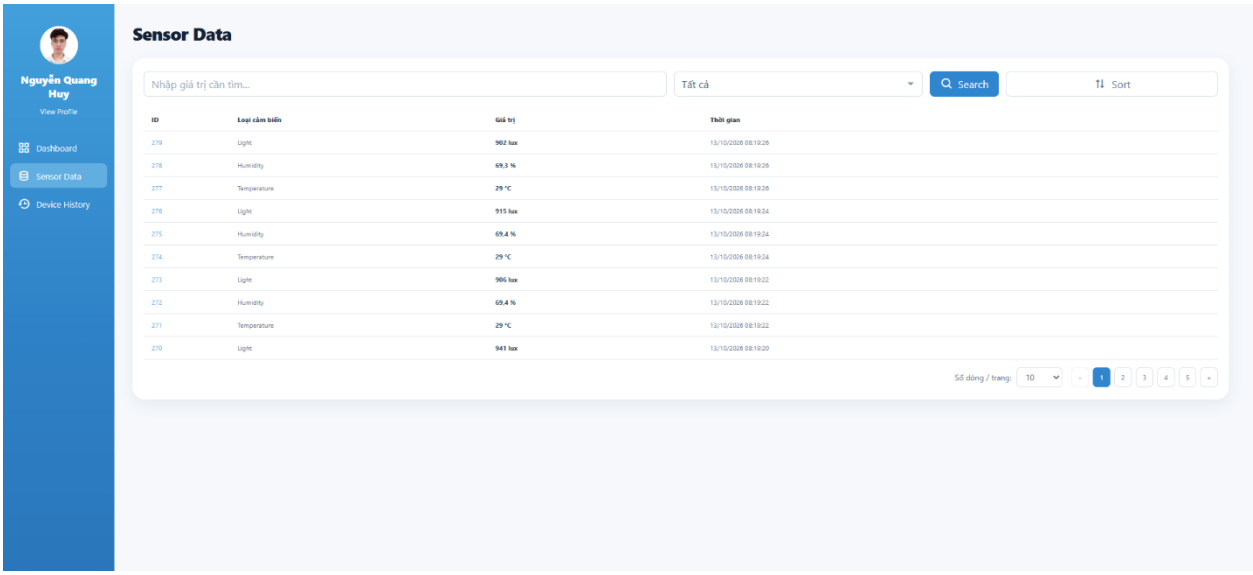
Hình 4.3. Kết quả giao diện Dashboard và điều khiển hai đèn

c. Màn hình Sensor Data

Sensor Data cung cấp bảng lịch sử các giá trị nhiệt độ, độ ẩm và ánh sáng được lưu trong MySQL. Người dùng có thể nhập nội dung tìm kiếm, chọn loại điều kiện và sắp xếp kết quả theo nhu cầu. Frontend gửi GET /api/sensor-data/history kèm các tham số searchType, value, page, size, sortBy và sortDirection; Backend lọc dữ liệu từ sensor_data và liên kết với sensors để xác định loại cảm biến.

Mỗi dòng kết quả thể hiện mã bản ghi, loại cảm biến, giá trị và thời điểm đo. Dữ liệu được chia trang để thuận tiện tra cứu khi lịch sử phát sinh nhiều bản ghi. Nếu không có dữ liệu tương ứng với điều kiện lọc, bảng hiển thị trạng thái rỗng; khi thay đổi trang hoặc thứ tự sắp xếp, nội dung bảng được truy vấn và cập nhật lại.

Chức năng chính: tìm kiếm bản ghi cảm biến; xem thời điểm và giá trị đo; sắp xếp dữ liệu; chuyển trang; xử lý trường hợp không có kết quả.



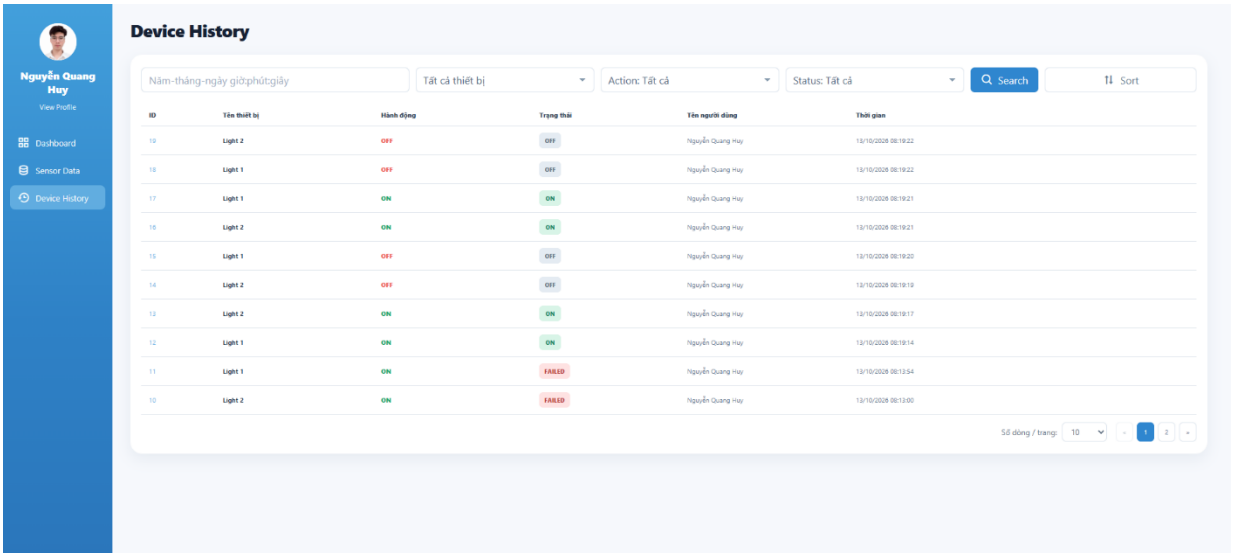
Hình 4.4. Kết quả màn hình Sensor Data

d. Màn hình Device Action History

Device Action History hiển thị các lần người dùng gửi lệnh bật/tắt thiết bị. Danh sách lịch sử được lấy từ GET /api/device-action/history; các điều kiện deviceId, action, status và time được kết hợp với tham số phân trang, sắp xếp để tra cứu bản ghi phù hợp. Bảng thể hiện người thực hiện, tên đèn, thao tác ON/OFF, trạng thái xử lý và thời điểm phát sinh.

Trạng thái trong lịch sử được trình bày tách biệt với hành động điều khiển. LOADING cho biết yêu cầu đang chờ phản hồi; ON hoặc OFF thể hiện trạng thái đã được thiết bị xác nhận; FAILED cho biết thao tác không nhận được xác nhận trong thời gian quy định hoặc xử lý không thành công. Việc lưu đầy đủ các trạng thái này giúp người dùng kiểm tra tiến trình điều khiển và xác định những lần thiết bị không phản hồi.

Chức năng chính: xem danh sách thao tác điều khiển; lọc theo thiết bị, hành động, trạng thái và thời gian; xem kết quả từng lệnh; phân trang, sắp xếp lịch sử.



Hình 4.5. Kết quả màn hình Device Action History

e. Màn hình Profile

Màn hình Profile trình bày thông tin cá nhân của người thực hiện và nhóm liên kết liên quan đến sản phẩm. Các mục thông tin được bố trí riêng với chức năng giám sát, giúp người xem nhanh chóng tìm đến tài liệu thiết kế, mã nguồn, tài liệu API hoặc những tài nguyên dự án được cung cấp. Giao diện này hoàn thiện phần thông tin chung của hệ thống mà không ảnh hưởng đến việc thu thập dữ liệu và điều khiển thiết bị.

Chức năng chính: hiển thị thông tin người thực hiện; tập hợp các đường dẫn phục vụ xem mã nguồn, thiết kế giao diện, API và tài liệu báo cáo.

4.3. Kết quả kiểm thử

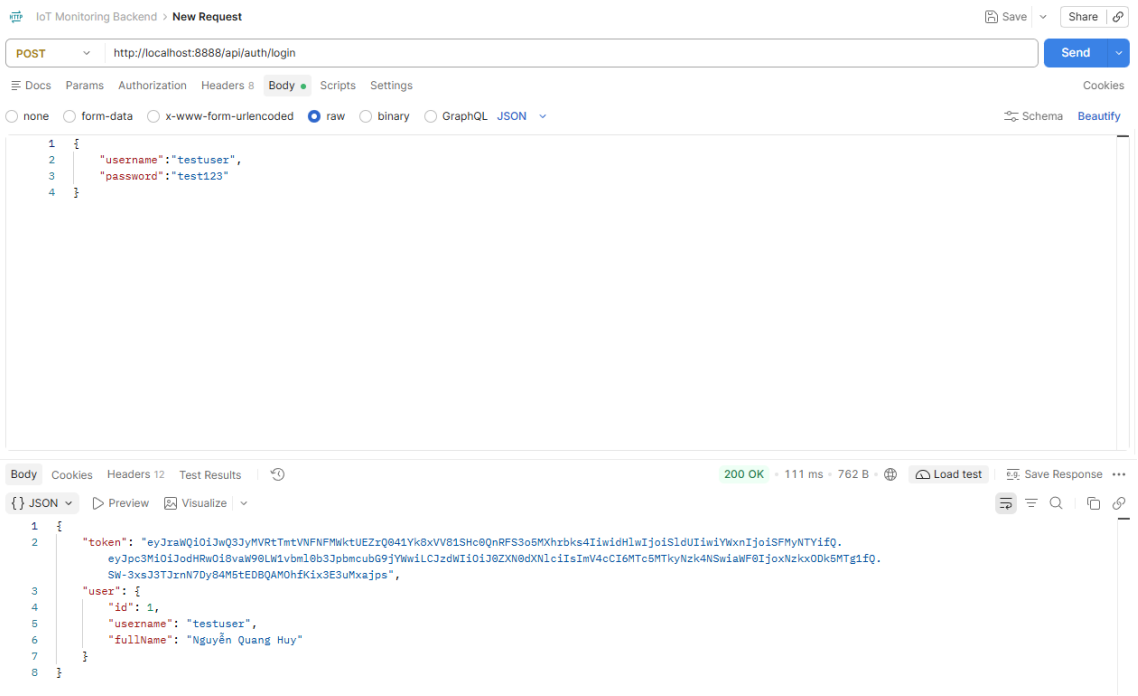
Các trường hợp kiểm thử được xây dựng cho những REST API trực tiếp phục vụ sáu use case, tập trung vào dữ liệu đầu vào, cấu trúc phản hồi và tình huống xử lý lỗi. Việc kiểm tra được thực hiện theo từng nhóm chức năng: đăng nhập, lấy dữ liệu cảm biến, hiển thị biểu đồ, điều khiển đèn và tra cứu lịch sử. Với thao tác điều khiển, cần đối chiếu thêm trạng thái trong MySQL với phản hồi từ ESP8266 qua MQTT để xác định kết quả cuối cùng.

a. Kiểm thử API đăng nhập - POST /api/auth/login

Mục tiêu kiểm thử: Kiểm tra việc xác thực tài khoản và định dạng dữ liệu mà Backend trả về cho giao diện đăng nhập.

Dữ liệu kiểm thử: Gửi yêu cầu POST chứa username và password hợp lệ; thực hiện thêm trường hợp sai mật khẩu hoặc bỏ trống một trường bắt buộc.

Kết quả đối chiếu: Với tài khoản hợp lệ, phản hồi gồm token và đối tượng user có id, username, fullName; yêu cầu không hợp lệ bị từ chối và giao diện hiển thị thông báo đăng nhập thất bại.



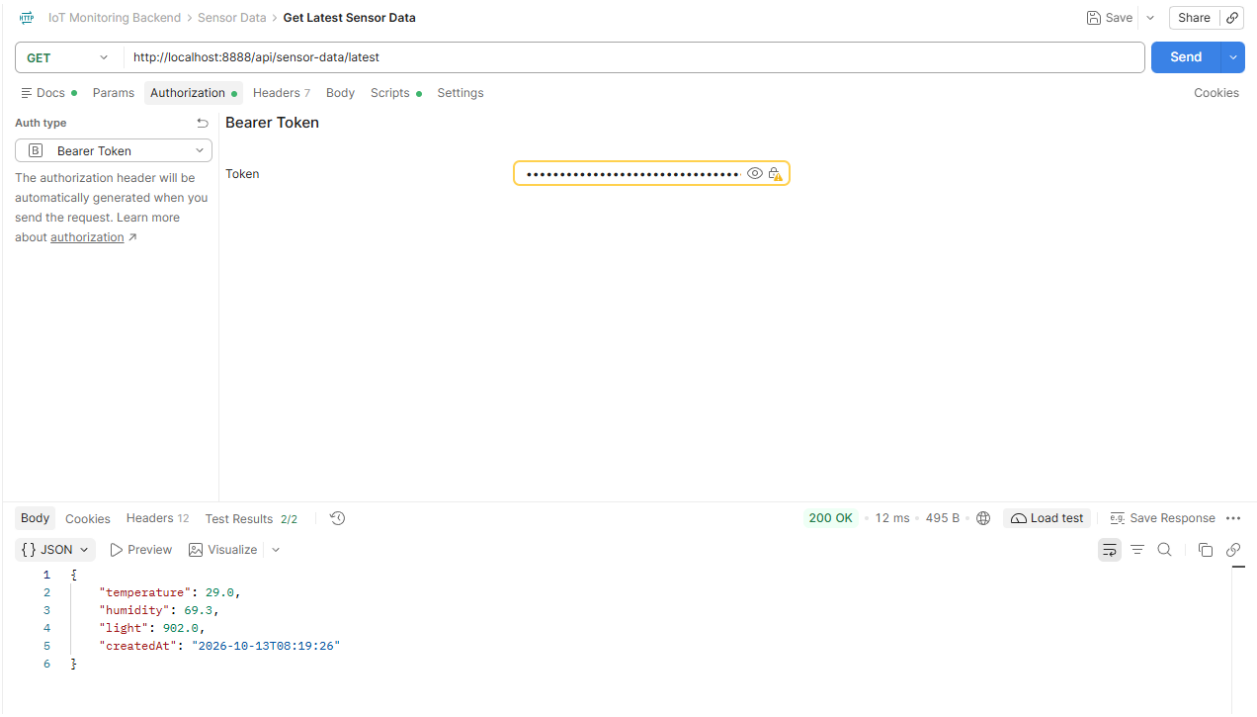
Hình 4.7. Kết quả kiểm thử API đăng nhập

b. Kiểm thử API lấy dữ liệu cảm biến mới nhất - GET /api/sensor-data/latest

Mục tiêu kiểm thử: Xác định khả năng cung cấp đồng thời ba thông số môi trường mới nhất để hiển thị trên Dashboard.

Dữ liệu kiểm thử: Gửi GET /api/sensor-data/latest không kèm body; so sánh các giá trị nhận được với bản ghi mới nhất của từng cảm biến trong sensor_data.

Kết quả đối chiếu: Phản hồi chứa temperature, humidity, light và createdAt; thông số và thời điểm cập nhật phải tương ứng với các dữ liệu được lưu gần nhất.



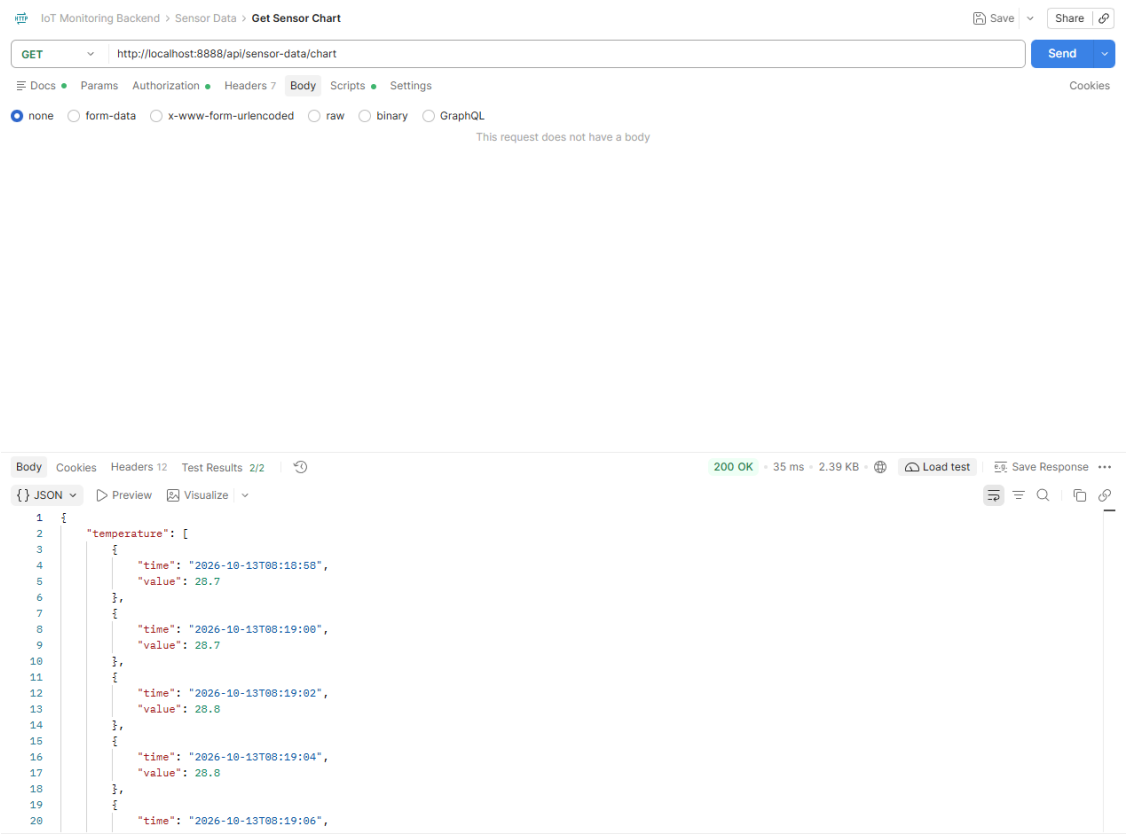
Hình 4.8. Kết quả kiểm thử API dữ liệu cảm biến mới nhất

c. Kiểm thử API dữ liệu biểu đồ - GET /api/sensor-data/chart

Mục tiêu kiểm thử: Kiểm tra khả năng lấy tập dữ liệu theo thời gian làm đầu vào cho biểu đồ nhiệt độ, độ ẩm và ánh sáng.

Dữ liệu kiểm thử: Gửi GET /api/sensor-data/chart; đối chiếu các chuỗi kết quả với 15 lần đo gần nhất được truy vấn từ cơ sở dữ liệu.

Kết quả đối chiếu: Dữ liệu trả về được chia thành temperature, humidity và light; mỗi phần tử chứa time và value. Thứ tự các điểm dữ liệu đáp ứng yêu cầu vẽ biểu đồ theo thời gian.



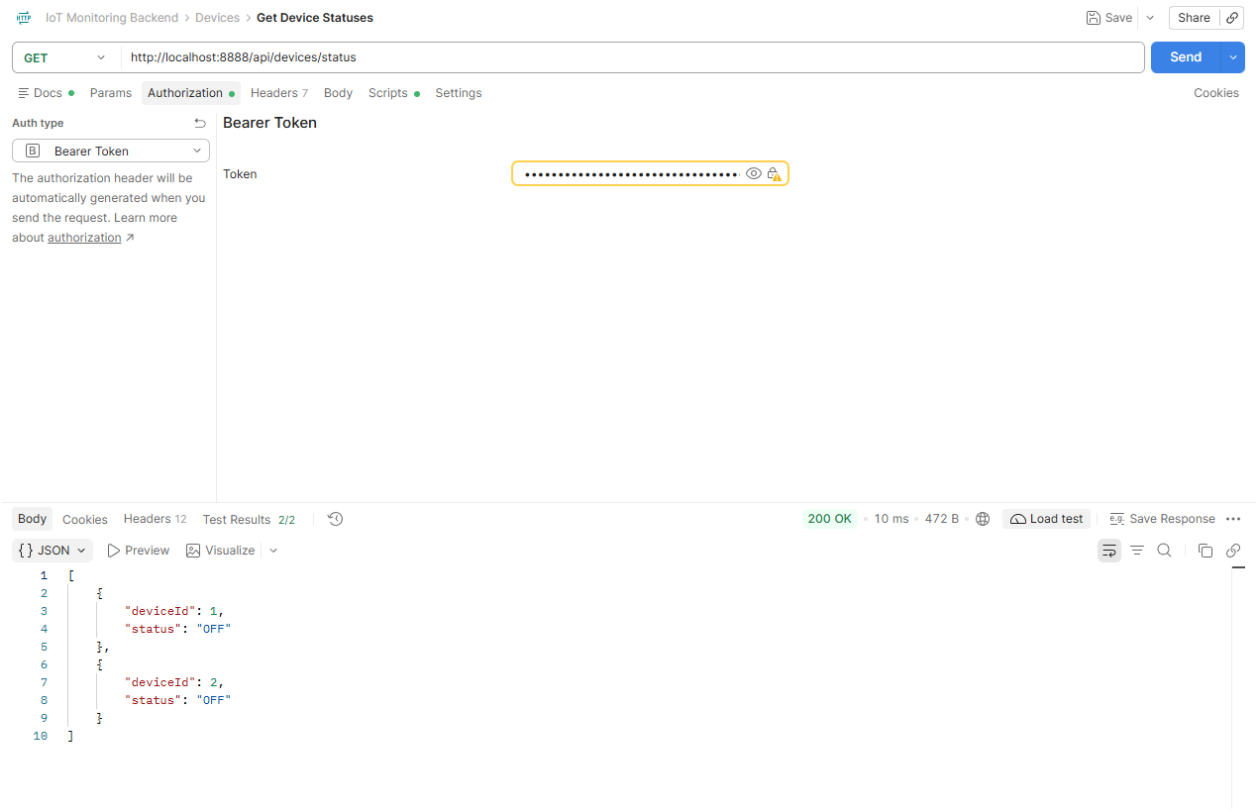
Hình 4.9. Kết quả kiểm thử API dữ liệu biểu đồ

d. Kiểm thử API lấy trạng thái thiết bị - GET /api/devices/status

Mục tiêu kiểm thử: Kiểm tra việc đọc trạng thái ON/OFF đã lưu của Đèn 1 và Đèn 2 trước khi người dùng thực hiện điều khiển.

Dữ liệu kiểm thử: Gửi GET /api/devices/status; đối chiếu deviceId và status trong phản hồi với các bản ghi tương ứng của bảng devices.

Kết quả đối chiếu: API trả danh sách trạng thái theo từng deviceId; dữ liệu hiển thị trên Dashboard cần trùng với trạng thái cuối cùng đã được Backend ghi nhận.



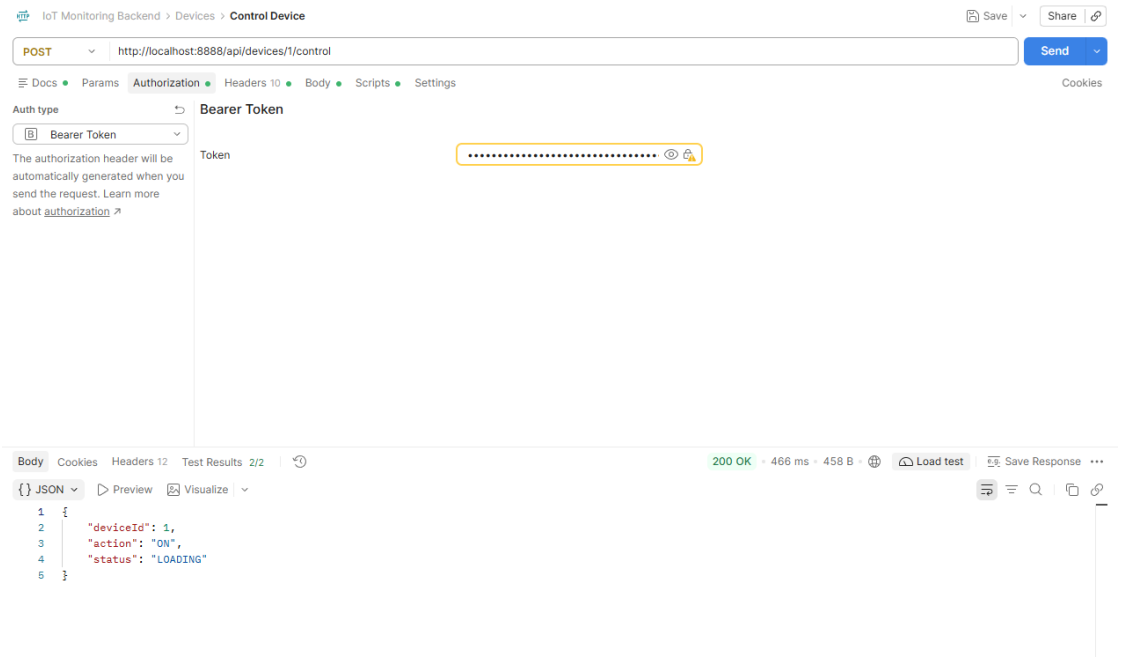
Hình 4.10. Kết quả kiểm thử API trạng thái thiết bị

e. Kiểm thử API điều khiển đèn - POST /api/devices/{deviceId}/control

Mục tiêu kiểm thử: Kiểm tra quá trình nhận lệnh ON/OFF, ghi lịch sử điều khiển và đồng bộ trạng thái thực tế của thiết bị.

Dữ liệu kiểm thử: Gửi POST /api/devices/1/control với body {"action":"ON"}; kiểm tra tương tự với OFF, thiết bị không tồn tại, hành động không hợp lệ và trường hợp không có phản hồi MQTT sau 5 giây.

Kết quả đối chiếu: Yêu cầu hợp lệ tạo bản ghi action_history với action tương ứng và status ban đầu là LOADING. Khi ESP8266 gửi phản hồi, action_history.status cùng devices.status được cập nhật thành ON/OFF; nếu quá thời gian chờ thì bản ghi chuyển FAILED và trạng thái cũ của thiết bị không



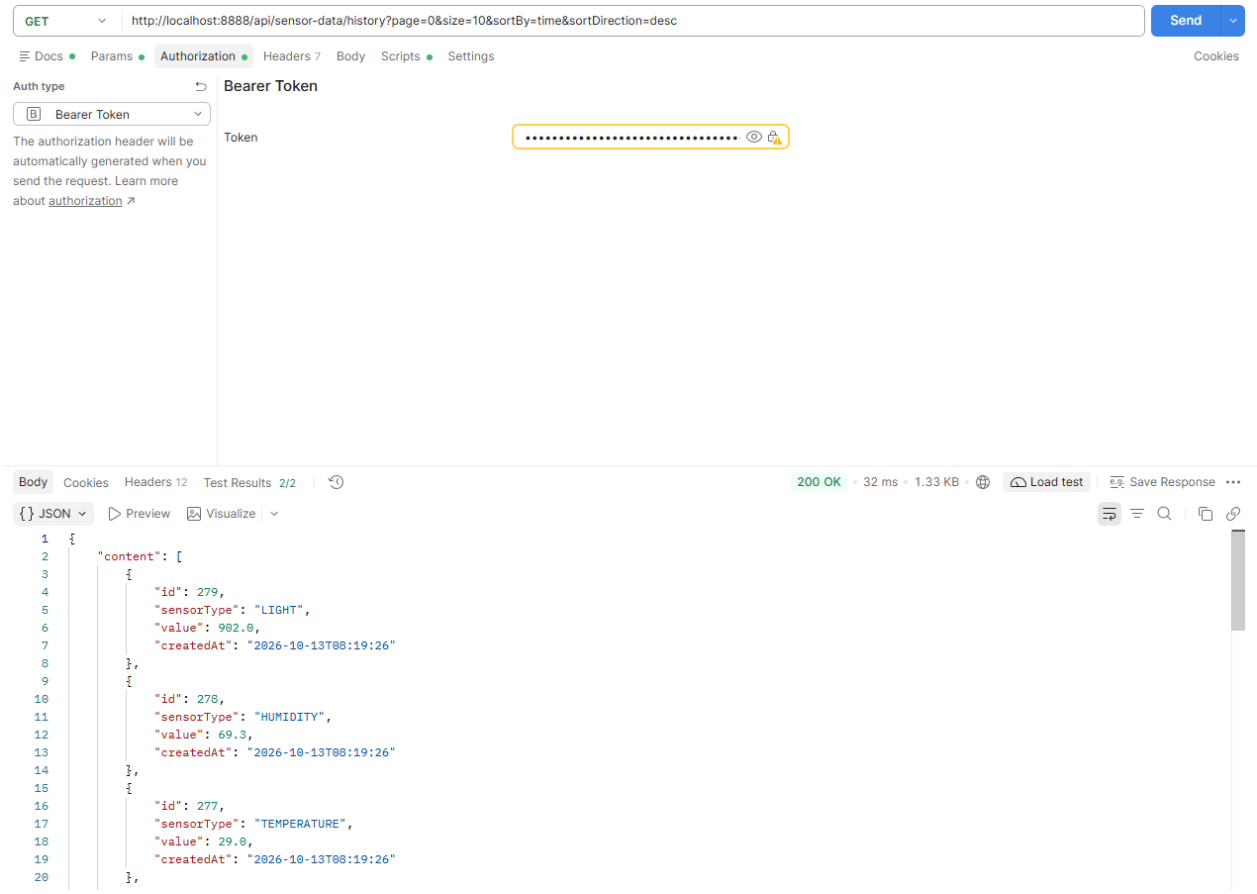
Hình 4.11. Kết quả kiểm thử API điều khiển đèn

f. Kiểm thử API lịch sử cảm biến - GET /api/sensor-data/history

Mục tiêu kiểm thử: Đánh giá khả năng trả dữ liệu cảm biến theo điều kiện tìm kiếm, thứ tự sắp xếp và trang được lựa chọn.

Dữ liệu kiểm thử: Gọi GET /api/sensor-data/history với searchType, value, page, size, sortBy và sortDirection; thử thêm điều kiện không có bản ghi phù hợp.

Kết quả đối chiếu: Dữ liệu phản hồi gồm content và thông tin phân trang; từng bản ghi có id, sensorType, value, createdAt. Số bản ghi và thứ tự hiển thị phải tương ứng với tham số truy vấn; bộ lọc không có kết quả trả về danh sách rỗng.



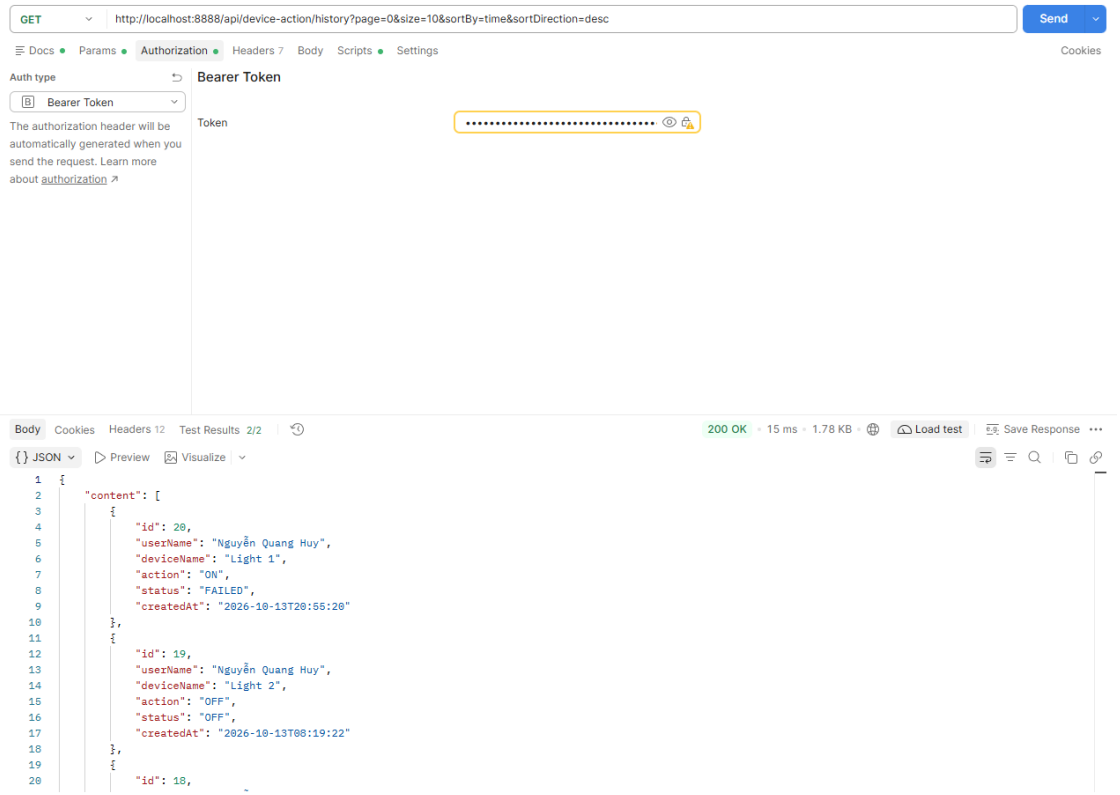
Hình 4.12. Kết quả kiểm thử API tra cứu lịch sử cảm biến

g. Kiểm thử API lịch sử điều khiển - GET /api/device-action/history

Mục tiêu kiểm thử: Kiểm tra việc truy xuất lịch sử bật/tắt đèn kèm thông tin người thao tác và trạng thái xử lý.

Dữ liệu kiểm thử: Gửi GET /api/device-action/history với deviceId, action, status, time, page, size, sortBy và sortDirection; đối chiếu các bản ghi nhận được với bảng action_history.

Kết quả đối chiếu: Phản hồi gồm danh sách content và thông tin phân trang, trong đó mỗi phần tử có id, userName, deviceName, action, status, createdAt. Các giá trị cần phản ánh đúng lịch sử xử lý, bao gồm LOADING, ON/OFF và FAILED khi có dữ liệu tương ứng.



Hình 4.13. Kết quả kiểm thử API tra cứu lịch sử điều khiển

4.4. Đánh giá kết quả

Hệ thống đáp ứng các yêu cầu chức năng được xây dựng từ sáu use case: xác thực người dùng, xem dữ liệu mới nhất, biểu diễn xu hướng, điều khiển hai đèn và tra cứu hai loại lịch sử. Việc tách Backend, MQTT Broker và ESP8266 tạo luồng trao đổi hai chiều rõ ràng; MySQL lưu dữ liệu phát sinh, còn WebSocket hỗ trợ cập nhật thông tin trên Dashboard. Các nội dung triển khai và kiểm tra được tổng hợp theo từng nhóm yêu cầu như sau:

Yêu cầu	Kết quả thực hiện	Đánh giá
Năm màn hình Web	Login, Dashboard, Sensor Data, Device Action History và Profile.	Đáp ứng
Sáu use case	Đăng nhập, dữ liệu thời gian thực, biểu đồ, điều khiển đèn và hai nhóm lịch sử.	Đáp ứng
Trao đổi dữ liệu	REST API, WebSocket và MQTT kết nối giao diện, Backend và ESP8266.	Đáp ứng
Cơ sở dữ liệu	Năm bảng cùng các quan hệ phục vụ dữ liệu đo và lịch sử thao tác.	Đáp ứng
Điều khiển và phản hồi	Theo dõi LOADING, ON/OFF và FAILED; phân biệt lệnh yêu cầu với trạng thái thực tế.	Đáp ứng
Kiểm thử chức năng	Bảy nhóm kiểm thử API, bao gồm dữ liệu hợp lệ và các tình huống ngoại lệ.	Cần đối chiếu minh chứng

Kết quả cho thấy kiến trúc được lựa chọn phù hợp với mô hình IoT giám sát và điều khiển ở quy mô nhỏ. Việc tách riêng dữ liệu đo, trạng thái hiện tại của thiết bị và lịch sử lệnh giúp kiểm tra được các tình huống phát sinh trong quá trình vận hành. Khi hoàn thiện sản phẩm, cần tiếp tục bổ sung minh chứng kiểm thử, theo dõi độ ổn định kết nối và đánh giá thời gian phản hồi để xác nhận chất lượng hoạt động của toàn hệ thống.

V. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Đề tài đã xây dựng mô hình hệ thống IoT giám sát nhiệt độ, độ ẩm và ánh sáng, kết hợp điều khiển hai đèn qua giao diện Web. Kiến trúc gồm ESP8266, MQTT Broker, Backend Java Spring Boot, MySQL và Frontend HTML/CSS/JavaScript. Dữ liệu được chuyển từ phần cứng đến Backend để lưu trữ, hiển thị theo thời gian thực và phục vụ truy vấn lịch sử; lệnh điều khiển được truyền theo chiều ngược lại và có phản hồi xác nhận từ thiết bị.

Các chức năng chính được thể hiện qua năm màn hình và sáu use case, bao gồm đăng nhập, giám sát dữ liệu mới nhất, xem biểu đồ, bật/tắt thiết bị và tra cứu lịch sử. Bên cạnh các chức năng người dùng, hệ thống còn có cơ chế phân biệt yêu cầu điều khiển với kết quả thực hiện, lưu lại trạng thái LOADING, ON/OFF hoặc FAILED. Cách xử lý này là nền tảng để theo dõi quá trình vận hành và phát hiện những lệnh không nhận được phản hồi.

5.2. Hạn chế của hệ thống

Hệ thống hiện được tổ chức cho một khu vực giám sát với ba nhóm thông số môi trường và hai thiết bị đèn. Việc quản lý nhiều phòng, nhiều vị trí lắp đặt hoặc nhiều nhóm người dùng chưa được tích hợp. Khả năng hoạt động khi số lượng thiết bị và dữ liệu đồng thời tăng cao cũng chưa được đánh giá bằng các phép thử tải chuyên sâu.

Hoạt động trao đổi dữ liệu phụ thuộc vào kết nối Wi-Fi và MQTT Broker, vì vậy mất mạng hoặc thiết bị ngắt kết nối có thể làm gián đoạn cập nhật thời gian thực. Các biện pháp bảo mật như HTTPS/WSS, xác thực kết nối MQTT, quản lý phiên đăng nhập và phân quyền truy cập cần được hoàn thiện khi triển khai trên mạng công cộng. Ngoài ra, chất lượng số liệu cảm biến cần được kiểm tra bằng phép đo thực tế để đánh giá sai số và độ ổn định trong quá trình sử dụng lâu dài.

5.3. Hướng phát triển

Hệ thống có thể được mở rộng theo hướng giám sát nhiều phòng hoặc khu vực. Cơ sở dữ liệu khi đó cần bổ sung thông tin vị trí lắp đặt, quan hệ giữa cảm biến và thiết bị với từng khu vực; giao diện Web có thêm chức năng chọn khu vực, quản lý thiết bị và phân quyền theo vai trò. Việc mở rộng số lượng cảm biến hoặc đèn vẫn có thể sử dụng kiến trúc MQTT hiện tại, đồng thời cần cơ chế định danh thiết bị để xử lý riêng từng nút IoT.

Đối với chức năng giám sát, có thể bổ sung ngưỡng cảnh báo khi nhiệt độ, độ ẩm hoặc ánh sáng vượt mức cho phép, gửi thông báo qua email và xây dựng báo cáo thống kê theo ngày hoặc tháng. Hệ thống cũng có thể tích hợp nhận dạng giọng nói để hỗ trợ điều khiển đèn bằng câu lệnh và phản hồi tình trạng hoạt động qua âm thanh. Dữ liệu lịch sử cảm biến và action_history còn là nguồn đầu vào để phân tích xu hướng sử dụng, phát hiện thời điểm bật/tắt bất thường và đưa ra cảnh báo sớm.

Để đáp ứng yêu cầu vận hành thực tế, cần tăng cường cơ chế tự kết nối lại khi mất MQTT, kiểm soát quyền truy cập API, mã hóa đường truyền, sao lưu MySQL và lưu nhật ký lỗi. Đồng thời, việc thử nghiệm với nhiều thiết bị và lượng dữ liệu lớn sẽ giúp xác định thời gian phản hồi, mức sử dụng tài nguyên và giới hạn mở rộng của hệ thống.

5.4. Kết luận

Hệ thống IoT giám sát môi trường và điều khiển đèn được xây dựng với sự phối hợp của phần cứng ESP8266, giao thức MQTT, Backend Spring Boot, MySQL và giao diện Web. Giải pháp cho phép tổ chức luồng dữ liệu cảm biến theo thời gian thực, lưu trữ thông tin lịch sử và thực hiện điều khiển thiết bị có xác nhận trạng thái. Các chức năng được phân tách thành những thành phần độc lập, tạo thuận lợi cho việc bảo trì và mở rộng.

Quá trình thực hiện giúp củng cố kiến thức về giao tiếp IoT, thiết kế cơ sở dữ liệu, REST API, WebSocket và mô hình ứng dụng nhiều lớp. Trên cơ sở những nội dung đã xây dựng, hệ thống có thể tiếp tục được cải thiện về độ tin cậy, khả năng bảo mật và các chức năng hỗ trợ giám sát thông minh để đáp ứng những yêu cầu sử dụng cao hơn.